A Lightweight External Memory Engine to Bridge the Capability Gap Between On-Device and Cloud Language Models

[Internal Project]

Ibrahim Sudheysi Ali

166344

ICS 4C

Supervisor Name

Tiberius Tabulu

An Informatics and Computer Science Project Report Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

November 2026

Declaration and Approval

I declare that this work has not been previously submitted and approved for the award of a degree by this or any other University. To the best of my knowledge and belief, the proposal contains no material previously published or written by another person except where due reference is made in the proposal itself.

Student Name: Ibrahim Sudheysi Ali

Admission Number: 166344

Student Signature: _____ Date: _____

The project report of Ibrahim Sudheysi Ali has been reviewed and approved by Tiberius Tabulu, School of Computing and Engineering Sciences, Strathmore University.

Supervisor Signature: _____ Date: _____

Acknowledgment

I thank God for the strength and good health that carried me through this work. I am grateful to my supervisor, Mr. Tiberius Tabulu, whose patient guidance and honest feedback shaped this project from the proposal to the finished system. I thank the faculty of the School of Computing and Engineering Sciences at Strathmore University for the foundation they gave me, and the Sprint II review panel for asking for something a person can see and use. Finally, I thank my family and friends for their encouragement and support throughout this journey.

Abstract

Cloud language models are capable, but every question costs money, needs a connection, and sends personal data off the device. Models that run on a consumer laptop are private, offline, and free per question. They are also weaker, most visibly on questions that need facts they have not memorised. Research indicates that this gap is largely missing knowledge, not missing reasoning ability.

This study designed, developed, and evaluated Burrow, a lightweight external memory engine that works beside a frozen on-device model. The memory is a vault of the person's own notes, documents, and code. On every question the engine decides whether the vault is needed, fits the amount of memory to the model within a token budget, and checks the sources an answer cites. Under the Design Science Research paradigm, the engine was built in Swift with an embedded database and delivered with a Mac application, a terminal chat, and a Model Context Protocol server. It was tested on 120 knowledge questions on three curated subjects and 50 reasoning questions from a public benchmark.

On the knowledge questions the on-device model, Gemma 4 12B, answered 9 alone and 102 with Burrow, while the cloud model, Claude, answered 40 alone and 109 with the same memory. On the reasoning questions the on-device model answered 47 alone, 7 with plain retrieval pasted in, and 48 with the engine, which left the memory out of all 50. A memory engine can therefore stand in for model scale on knowledge-bound tasks without harming reasoning.

Keywords: on-device language models, external memory, retrieval-augmented generation, capability gap, token budget

Table of Contents

Declaration and Approval ii
Acknowledgment iii
Abstract iv
Table of Contents v
List of Figures viii
List of Tables x
List of Equations xi
List of Abbreviations xii
Chapter 1: Introduction 1
1.1 Background Information 1
1.2 Problem Statement 1
1.3 Research Objectives 2
1.3.1 General Objective 2
1.3.2 Specific Objectives 2
1.4 Research Questions 2
1.5 Justification 2
1.6 Scope 3
1.7 Limitations and Delimitations 3
1.7.1 Limitations 3
1.7.2 Delimitations 3
Chapter 2: Literature Review 4
2.1 Introduction 4
2.2 Current State of On-Device Language Models and External Memory 4
2.2.1 Overview of On-Device Language Models 4
2.2.2 The Capability Gap and Long-Tail Knowledge 4
2.2.3 Retrieval and Agent Memory 4
2.2.4 Key Challenges 5
2.3 Related Works 5
2.3.1 MemGPT 5
2.3.2 Generative Agents 6
2.3.3 MemoryBank 6
2.3.4 HippoRAG 7
2.3.5 Mem0 7
2.3.6 Chroma 8
2.3.7 Obsidian 9
2.3.8 Claude Code and the Model Context Protocol 9
2.4 Gaps in Related Works 10
2.5 Conceptual Framework 11
Chapter 3: Methodology 12
3.1 Introduction 12
3.2 Research Paradigm 12
3.2.1 Data Acquisition 12
3.2.2 Data Processing 12
3.2.3 Model Training 12
3.2.4 Model Validation and Testing 13
3.3 System Development Methodology 13
3.3.1 Justification 13
3.4 Methodology Diagram 14

3.4.1 Increment 1: Evaluation Harness 14
3.4.2 Increment 2: Simplest Treatment 14
3.4.3 Increment 3: Engine Steps and Skills 14
3.4.4 Increment 4: Device Profiling 14
3.4.5 Increment 5: Application, Terminal Chat, and MCP Server 15
3.5 System Analysis Diagrams 15
3.5.1 Use Case Diagram 15
3.5.2 Sequence Diagram 15
3.5.3 Activity Diagram 15
3.5.4 Class Diagram 15
3.6 System Design Diagrams 15
3.6.1 Database Schema 15
3.6.2 Wireframes 15
3.6.3 System Architecture 16
3.7 Deliverables 16
3.7.1 The Burrow Engine Library and Command-Line Tool 16
3.7.2 The Mac Application and Terminal Chat 16
3.7.3 The MCP Server 16
3.7.4 The Evaluation Harness 16
3.7.5 Results and Documentation 16
3.8 Tools and Techniques 16
3.8.1 Swift 16
3.8.2 SQLite with FTS5 16
3.8.3 Ollama and the Local Models 17
3.8.4 The Hosted API 17
3.8.5 The Embedding Model 17
3.8.6 Chroma 17
3.8.7 LangChain 17
3.8.8 Python 17
3.8.9 Git and GitHub 17
Chapter 4: System Analysis and Design 18
4.1 Introduction 18
4.2 System Requirements 18
4.2.1 Functional Requirements 18
4.2.2 Non-Functional Requirements 18
4.3 System Analysis Diagrams 19
4.3.1 Use Case Diagram 19
4.3.2 Use Case Description 20
4.3.3 Sequence Diagram 20
4.3.4 Activity Diagram 22
4.3.5 Class Diagram 23
4.4 System Design Diagrams 24
4.4.1 System Architecture 24
4.4.2 Database Schema 26
4.4.3 Wireframes 26
4.4.4 Deployment Diagram 28
Chapter 5: System Implementation and Testing 30
5.1 Introduction 30
5.2 Implementation Environment 30
5.2.1 Hardware Specifications 30

5.2.2 Software Specifications 30
5.3 Description of the Dataset 31
5.4 System Implementation 31
5.4.1 Decide 31
5.4.2 Fit 32
5.4.3 Answer and Check 33
5.4.4 Skills and Learn 34
5.4.5 Serving Other Models and Tools 34
5.5 Experiments and Evaluation 35
5.5.1 Experimental Set-ups 35
5.5.2 Evaluation Metrics 36
5.5.3 Results 36
5.5.4 A Coding Trial Against the Cloud Model 40
5.6 System Testing 41
5.7 Results and Discussion 42
5.8 GitHub Documentation 43
References 45
Appendices 52
Appendix 1: Gantt Chart 52
Appendix 2: Feedback-Resolution Matrix 53
Appendix 3: Further Results 56
Appendix 4: Requirements Traceability 58
Appendix 5: User Guide 59
Appendix 6: More Screens of the App 60
Appendix 7: AI-Use Log 61
Appendix 8: Similarity Report 62

List of Figures

Figure 2.1: The MemGPT virtual context architecture. Adapted from Packer et al. (2023). 6
Figure 2.2: The Smallville environment of Generative Agents. Source: Park et al. (2023). 6
Figure 2.3: The MemoryBank framework with updating and forgetting. Source: Zhong et al. (2024). 7
Figure 2.4: HippoRAG offline indexing and online retrieval. Source: HippoRAG project repository (Gutiérrez et al., 2024). 7
Figure 2.5: The Mem0 memory layer within an agent stack. Source: Mem0 documentation (Chhikara et al., 2025). 8
Figure 2.6: The architecture of distributed Chroma. Source: Chroma (2026), https://docs.trychroma.com. 8
Figure 2.7: The graph view of an Obsidian vault. Source: Obsidian (2025b), https://help.obsidian.md/plugins/graph. 9
Figure 2.8: The Model Context Protocol between AI applications and data sources. Source: Model Context Protocol (2026), https://modelcontextprotocol.io. 10
Figure 2.9: Conceptual framework of the study. 11
Figure 3.1: Incremental development of the Burrow engine. 14
Figure 4.1: Use case diagram of Burrow. 19
Figure 4.2: Sequence diagram of a question asked through the engine. 21
Figure 4.3: Sequence diagram of an outside model preparing a task through MCP. 22
Figure 4.4: Activity diagram of the engine's loop. 23
Figure 4.5: Class diagram of the engine. 24
Figure 4.6: Architecture of Burrow. 25
Figure 4.7: The five steps of the engine and the mechanisms of the proposal. 25
Figure 4.8: Logical schema of the index. 26
Figure 4.9: Wireframe of the main window with the Ask panel. 27
Figure 4.10: Wireframe of the note editor with its links. 27
Figure 4.11: Wireframe of the step where the person keeps what was decided. 28
Figure 4.12: Wireframe of the terminal chat. 28
Figure 4.13: Deployment diagram of Burrow. 29
Figure 5.1: The engine answering a question about code and reporting its decision, the skill used, the files read and the citation check. 32
Figure 5.2: A sum in the Mac app. The line under the answer states that the vault was left out and which method was used. 32
Figure 5.3: Measuring a model once with `burrow fit`, and the skills in the vault. 33
Figure 5.4: A question answered from the vault in the Mac app, with the engine's decision and the files read. 33
Figure 5.5: A skill as it is stored in the vault. 34
Figure 5.6: An outside client calling the tool `vault_prepare` through the Model Context Protocol. 35
Figure 5.7: Burrow as the retriever in a LangChain pipeline. 35
Figure 5.8: Answers right by set-up and task type, Gemma 4 12B and the cloud model. 37
Figure 5.9: Answers right by set-up and task type, Qwen3.5 4B and the cloud model. 39
Figure 5.10: Knowledge answers right against tokens read, as the amount of the vault shown to Gemma 4 12B grows. 39
Figure 5.11: Median time of one lookup on the Django index, before and after the search was rebuilt. 40
Figure 5.12: The same three briefs built by Gemma 4 12B alone, by Gemma with a skill and a note of known failures, and by Claude alone. 41

Figure 5.13: The repository's recent history, tags and folders. 44
Figure A1.1: Project work plan to the end of November 2026. 52
Figure A6.1: A note being edited, with its links and properties. 60
Figure A6.2: The graph of a vault, coloured by folder. 60
Figure A6.3: A note in the reading view. 61

List of Tables

Table 2.1: Comparison of the related works 10
Table 4.1: Functional requirements 18
Table 4.2: Non-functional requirements 19
Table 4.3: Use case description: Ask a question 20
Table 5.1: Hardware specifications 30
Table 5.2: Software specifications 30
Table 5.3: The evaluation vault and its questions 31
Table 5.4: Answers right by set-up and task type, Gemma 4 12B 37
Table 5.5: Fraction of the capability gap closed (Equation 3.1), against the cloud model alone 37
Table 5.6: Cost of an answer on knowledge questions: tokens read, seconds, and whether the evidence was retrieved 38
Table 5.7: How much of the vault the model is shown, against what it gains, Gemma 4 12B, knowledge questions 40
Table 5.8: System test cases 41
Table A1.1: Course milestones and their status 52
Table A2.1: Feedback received and its resolution 53
Table A2.2: Changes to Chapters 1 to 3 since the approved proposal 53
Table A3.1: Answers right by set-up and task type, Qwen3.5 4B 56
Table A3.2: Answers right by subject, Gemma 4 12B and the cloud model 56
Table A3.3: Paired comparisons by exact sign test 57
Table A3.4: What the engine decided, Gemma 4 12B 57
Table A3.5: The method handed to the model, Gemma 4 12B 57
Table A3.6: Wrong knowledge answers, by whether the files holding the answer were shown to the model 58
Table A3.7: One question of each kind with its answer key 58
Table A4.1: Traceability from requirement to component and test 59
Table A7.1: AI-use log 61

List of Equations

Equation 3.1: Fraction of the capability gap closed. 13
Equation 4.1: Cosine similarity between a question and a piece. 26
Equation 4.2: Merged score of a piece 26
Equation 4.3: Score of a neighbouring file during a walk. 26
Equation 5.1: Accuracy of a set-up. 36
Equation 5.2: Exact two-sided sign test on paired answers. 36

List of Abbreviations

AI - Artificial Intelligence
API - Application Programming Interface
BGE - BAAI General Embedding
BM25 - Best Matching 25
CPU - Central Processing Unit
DSR - Design Science Research
EM - Exact Match
F1 - F-Score (Harmonic Mean of Precision and Recall)
FTS5 - Full-Text Search version 5 (SQLite)
GPU - Graphics Processing Unit
GSM8K - Grade School Math 8K
JSON - JavaScript Object Notation
KG - Knowledge Graph
LLM - Large Language Model
MCP - Model Context Protocol
OD - On-Device
PPR - Personalized PageRank
QA - Question Answering
RAG - Retrieval-Augmented Generation
RAM - Random Access Memory
RRF - Reciprocal Rank Fusion
SLM - Small Language Model
SQL - Structured Query Language
SWE-bench - Software Engineering Benchmark

Chapter 1: Introduction

1.1 Background Information

Language models now sit behind most conversational software. Kaplan et al. (2020) showed that the capability of a language model grows predictably with its size and its training data. The strongest models are therefore the largest, and they run as cloud services. Every question sent to them travels over a network and leaves the person's device. Z. Zhang et al. (2024) studied real conversations with a cloud assistant and interviewed 19 of its users. They found that people constantly trade privacy against usefulness, and that wrong mental models limit their grasp of the risk.

A second family of models runs directly on consumer hardware. J. Xu et al. (2024) reviewed these on-device models and gave three reasons for running a model locally: lower latency, data that stays on the device, and personalisation. Lu et al. (2024) measured 70 open models of up to five billion parameters and reported both what they can do and what they cost to run on a device. The open-weight families released in 2026, Gemma 4 (Gemma Team, 2026) and Qwen3.5 (Qwen Team, 2026), include models sized for a laptop.

The weakness of a small model is concentrated in what it knows. Mallen et al. (2023) showed that language models struggle with less popular facts and that scaling a model up barely helps on this long tail. Kandpal et al. (2023) explained why. A model's accuracy on a factual question depends on how many training documents mention the fact, and a model would need to grow by many orders of magnitude to answer rarely supported questions well. A person's own notes, documents, and code are the extreme case, because no model saw them in training.

Knowledge of this kind can be supplied from outside the model. Lewis et al. (2020) introduced retrieval-augmented generation, which pairs a model with an index of documents that it reads at answer time. Soudani et al. (2024) compared this approach with fine-tuning across twelve language models and found that retrieval won by a large margin on the least popular facts. If much of the gap between small and large models is missing knowledge, an external memory might recover a meaningful share of it without changing the model.

1.2 Problem Statement

A person who wants a private, offline, low-cost assistant must accept a small on-device model. With it comes a marked drop in capability, most visibly on questions that need facts the model has not memorised. The two standard remedies do not fit the device. A larger model no longer fits in the machine's memory. Fine-tuning the small model needs extensive resources (Soudani et al., 2024) and must be repeated whenever the knowledge changes.

Retrieval is the third path, but pasting retrieved text into every prompt is not safe. Yoran et al. (2024) found across five question-answering benchmarks that retrieval sometimes lowered accuracy when the retrieved text was irrelevant. Shi et al. (2023) added irrelevant information to grade-school mathematics problems and saw model performance fall dramatically. Levy et al. (2024) showed that reasoning drops at input lengths far shorter than a model's technical maximum. A memory beside a small model must therefore decide when to supply text and how much. Published remedies fine-tune the model or train a classifier (Jeong et al., 2024; Yoran et al., 2024).

It is also not known how much of the gap such a memory closes. Narayan et al. (2025) came nearest: a small on-device model that read local documents recovered 87% of a cloud model's performance, but only by working with the cloud model. No study found in the literature measures what memory alone achieves on a consumer device, by task type, and at what cost. This study therefore developed Burrow, a lightweight external memory engine that sits beside

a frozen on-device model and gives it a vault of the person's own notes, documents, and code. The study then measured how much of the capability gap this memory closed, separately for knowledge-bound and reasoning-bound tasks, and at what cost in tokens and latency.

1.3 Research Objectives

1.3.1 General Objective

The general objective of this study was to design, develop, and validate a lightweight memory engine for a frozen small language model on a consumer laptop, in order to quantify how much of the capability gap between small on-device and large cloud language models such a memory engine can close, by task type, under explicit token and latency accounting.

1.3.2 Specific Objectives

i. To investigate the capability gaps between large, small, and on-device language models, together with existing memory, retrieval, and compression techniques and the interaction between a model's internal (parametric) and external (retrieved) knowledge, so as to establish their applicability and limitations in the on-device setting.
ii. To design a lightweight memory engine for a frozen small language model, comprising a write policy that distils interactions into compact stored notes, a retrieval policy that ranks stored passages by word match and by meaning and follows the links between files, and a token budgeting mechanism that controls how much memory the model reads.
iii. To develop the designed memory engine in Swift on an Apple-silicon consumer laptop with no dedicated graphics card, using an embedded database as the index of the memory store.
iv. To evaluate the implemented engine experimentally by measuring the fraction of the capability gap between a small local model and a large cloud model that the engine closes, separately for knowledge-bound and reasoning-bound tasks, together with the token cost and latency of every answer.
v. To determine the token budget that maximises the fraction of the capability gap closed per token, so that the engine can be configured for the best accuracy-to-cost trade-off on a resource-constrained device.

1.4 Research Questions

i. What memory, retrieval, and compression techniques exist for language models, and what are their limitations in the on-device setting?
ii. How much of the capability gap between a small on-device language model and a large cloud language model can a frozen-weights memory engine close?
iii. Does the memory engine help more on knowledge-bound questions than on reasoning-bound questions?
iv. What context token budget maximises the fraction of the capability gap closed per token, and how does gap closure scale as the budget increases?
v. What is the cost of the memory engine in tokens and latency when measured on a consumer laptop?

1.5 Justification

The study serves three groups. People who handle private material gain an assistant that keeps it on their own machine. Mireshghallah et al. (2024) found personal information in 16% of code-editing conversations with commercial chat models, which is the kind of material a vault

of notes and code holds. In Kenya, personal data falls under the Data Protection Act (2019), and Ogonjo (2025) noted that data and AI sovereignty are reflected in the national AI strategy. Developers of on-device assistants gain a measured account of how far memory engineering can stand in for model scale. Researchers gain an evaluation method, the gap-closed measure reported per task type with token and latency accounting on consumer hardware, against which later memory systems can be compared.

1.6 Scope

The study targeted one consumer laptop, an Apple MacBook Pro with an M5 Pro chip, 24 gigabytes of shared memory, and no dedicated graphics card. The on-device models were open-weight models run locally through Ollama: Gemma 4 12B as the main model and Qwen3.5 4B as the small model. The cloud model, Claude, served only as the upper reference. The evaluation used English questions of two kinds: knowledge questions on three curated subjects that the on-device model cannot know well, and mathematical reasoning questions from a public benchmark. The system delivered was the memory engine, its evaluation harness, and four ways to use it: a command-line tool, a native Mac application, a terminal chat, and a Model Context Protocol server.

1.7 Limitations and Delimitations

1.7.1 Limitations

Four constraints outside the study's control shape how the findings should be read.

i. Latency was measured on one consumer laptop and may differ on other machines.
ii. The cloud reference model is a hosted service whose behaviour may change over time, which can move the ceiling used to calculate the capability gap.
iii. The questions on the curated subjects were written by a language model from the source documents and not by people, so the results may not transfer equally to every live use.
iv. The cost and time available to one undergraduate project limited how many models, devices, languages, and modalities could be tested.

1.7.2 Delimitations

The study narrowed its focus on purpose. It fixed one laptop as the target device and reported the full hardware specification and the seconds taken by every answer. It pinned and logged one version of the cloud model for the whole experiment. It used three unrelated subjects and one public benchmark, and it reported single-fact, multi-fact, and mathematical reasoning results separately. It did not train or fine-tune any language model, did not port the engine to mobile phones, and did not evaluate multilingual or multimodal memory. The graph-based retrieval layer that the proposal held back as optional was built in a light form, as a walk over the links that already exist between files.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the research that the study built on. Section 2.2 describes the current state of on-device language models and external memory: the models themselves, the gap that separates them from cloud models, retrieval and agent memory, and the challenges that remain. Section 2.3 examines eight related systems, five that implement memory for language models and three that Burrow was compared with or built to work beside. Section 2.4 states the gaps those systems leave. Section 2.5 presents the conceptual framework that guided the study.

2.2 Current State of On-Device Language Models and External Memory

2.2.1 Overview of On-Device Language Models

Kaplan et al. (2020) established that the capability of a language model grows smoothly with its parameter count and its training data, which pushed the field toward large models served from the cloud. A counter-movement followed. Qwen2.5 (Yang et al., 2024), Llama 3.2 (Grattafiori et al., 2024), and Phi-3 (Abdin et al., 2024) were published as open-weight models small enough for an ordinary laptop, and Gunter et al. (2024) described the model that Apple ships on its own devices. J. Xu et al. (2024) reviewed this movement and named lower latency, data that stays local, and personalisation as its reasons.

Each generation moves the same quality into a smaller model. Z. Liu et al. (2024) showed that careful architecture lets models under one billion parameters come close to a seven-billion-parameter model on API-calling tasks. The Gemma 3 report stated that its 4B model was competitive with the previous generation's 27B model (Gemma Team, 2025). The families released in 2026, Gemma 4 (Gemma Team, 2026) and Qwen3.5 (Qwen Team, 2026), include the two models hosted in this study. Lu et al. (2024) measured 70 open small models and reported their ability together with their latency and memory on a device.

2.2.2 The Capability Gap and Long-Tail Knowledge

The evidence most important to this study concerns what the gap is made of. Mallen et al. (2023) tested ten models on 14,000 factual questions and found that models struggle with less popular facts and that scaling barely helps in the long tail. Sun et al. (2024) saw the same pattern in 16 public models, which were far from perfect on the rarer entities. Kandpal et al. (2023) traced the cause to training data. Accuracy on a fact depends on how many training documents mention it, and closing the long tail by size alone would need many orders of magnitude more parameters.

Retrieval reduces that dependence. In the study by Mallen et al. (2023), retrieval-augmented models largely outperformed models that were orders of magnitude larger. Soudani et al. (2024) found across twelve models that retrieval beat fine-tuning by a large margin on the least popular facts. Missing knowledge can therefore be supplied from outside a model. This study treated knowledge-bound and reasoning-bound tasks separately for that reason.

2.2.3 Retrieval and Agent Memory

Lewis et al. (2020) introduced retrieval-augmented generation, which pairs a generator with an index of retrieved documents, and Fan et al. (2024) surveyed the architectures that followed. Two inexpensive rankings underlie most retrieval. Ranking by word match uses the BM25 function (Robertson & Zaragoza, 2009), and ranking by meaning compares embedding vectors

(Karpukhin et al., 2020). Reciprocal rank fusion merges the two lists without training (Cormack et al., 2009). Bruch et al. (2023) showed that a tuned weighted sum of scores can outperform this fusion, but it needs a training set.

Memory extends retrieval from documents to what an agent keeps over time. Sumers et al. (2024) described a language agent as having working, episodic, semantic, and procedural memory. Hu, Liu, et al. (2025) separated agent memory from retrieval-augmented generation and classified it by form, function, and dynamics. Huang et al. (2026) treated portable agent skills as part of memory, and Luo et al. (2026) described three stages of agent memory: storage, reflection, and experience. In these terms Burrow's vault is semantic memory, its decision notes are episodic memory, and its skills are procedural memory.

2.2.4 Key Challenges

Four challenges remain. The first is that retrieved text can harm an answer. Yoran et al. (2024) found that retrieval sometimes lowered accuracy across five question-answering benchmarks. Cuconasu et al. (2024) identified the harmful passages as those a retriever scores highly although they do not contain the answer. Reasoning tasks are especially exposed. Shi et al. (2023) saw performance fall dramatically when irrelevant information was added to grade-school mathematics problems, and Mirzadeh et al. (2025) reported drops of up to 65% from a single clause that only seemed relevant.

The second is that more context is not better. Levy et al. (2024) found a notable drop in reasoning at input lengths far shorter than a model's maximum. Hsieh et al. (2024) tested 17 long-context models and found that only half kept satisfactory performance at 32,000 tokens. Jin et al. (2025) showed that output quality first improves and then declines as the number of retrieved passages rises.

The third is trust in what a model does with its context. Joren et al. (2025) found that smaller models often hallucinate or abstain even when the context is sufficient. T. Gao et al. (2023) found that even the best models lacked complete citation support half of the time on one benchmark.

The fourth is that a memory which accepts writes can be attacked. Greshake et al. (2023) showed that instructions hidden in retrieved data can take over an application. Dong et al. (2025) showed that an attacker who can only send queries can plant malicious records in an agent's memory. These findings motivated two design choices in Burrow: every proposed note is backed by the person's own words, and nothing is written to the vault without consent.

2.3 Related Works

2.3.1 MemGPT

Packer et al. (2023) treated the memory problem as an operating system would. In MemGPT the model's context window is the fast main memory and an external store is the disk, and the system pages information in and out as the conversation demands. This lets a model with a fixed window sustain document analysis and multi-session chat. The design made clear that what a model reads at each step is a policy decision. Later systems took that decision for each question: Adaptive-RAG routes a question to no retrieval, one-step retrieval, or multi-step retrieval (Jeong et al., 2024). MemGPT manages context length, however. It does not measure capability gain for a small model, or token cost and latency on a device. Figure 2.1 shows the two-tier architecture.

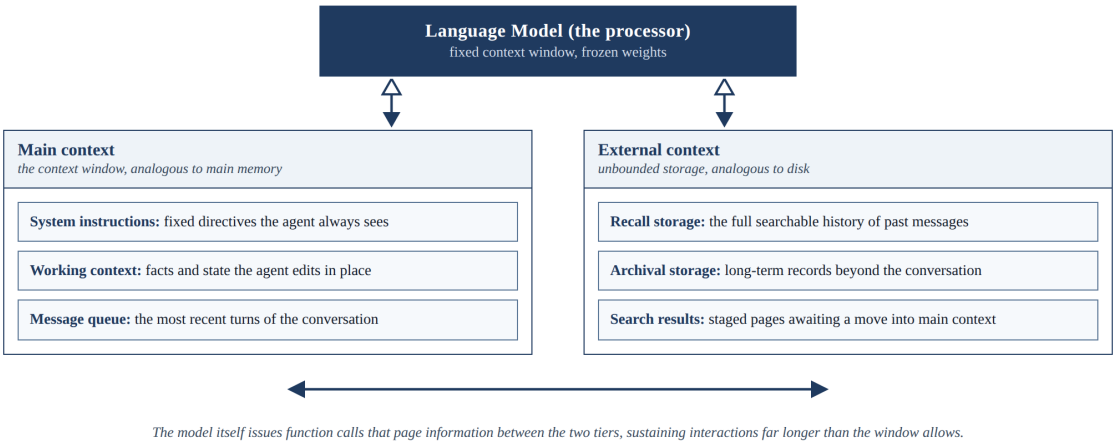


Figure 2.1: The MemGPT virtual context architecture. Adapted from Packer et al. (2023).

2.3.2 Generative Agents

Park et al. (2023) gave simulated agents a memory stream of timestamped observations written in natural language. Each agent retrieved from its stream by combining three scores: relevance to the current situation, recency, and importance. With this memory the agents produced believable long-running behaviour in a simulated town, and the three-part score shaped the retrieval policy first proposed for this study. Zhao et al. (2024) later showed that insights written in natural language and recalled at inference improve an agent without any parameter updates. Generative Agents was designed for simulation and not for question answering, however, and its memory records are verbose. It leaves open how a memory should be read when every token costs time on a device. Figure 2.2 shows the town in which the agents live.



Figure 2.2: The Smallville environment of Generative Agents. Source: Park et al. (2023).

2.3.3 MemoryBank

Zhong et al. (2024) added a model of forgetting to long-term memory. MemoryBank strengthens or weakens each stored memory over time in the spirit of the Ebbinghaus forgetting curve. What the system keeps therefore reflects both how often a memory is used and how old it is. The contribution is a principled answer to the question of what should be kept. Hu, Wang, and McAuley (2025) later named selective forgetting as one of four competencies of a memory

agent. MemoryBank was evaluated on the quality of long-term companionship behaviour. It does not report capability gain against a larger model, token cost, or latency on a device. Figure 2.3 shows the memory bank, the forgetting curve, and retrieval into the conversation.

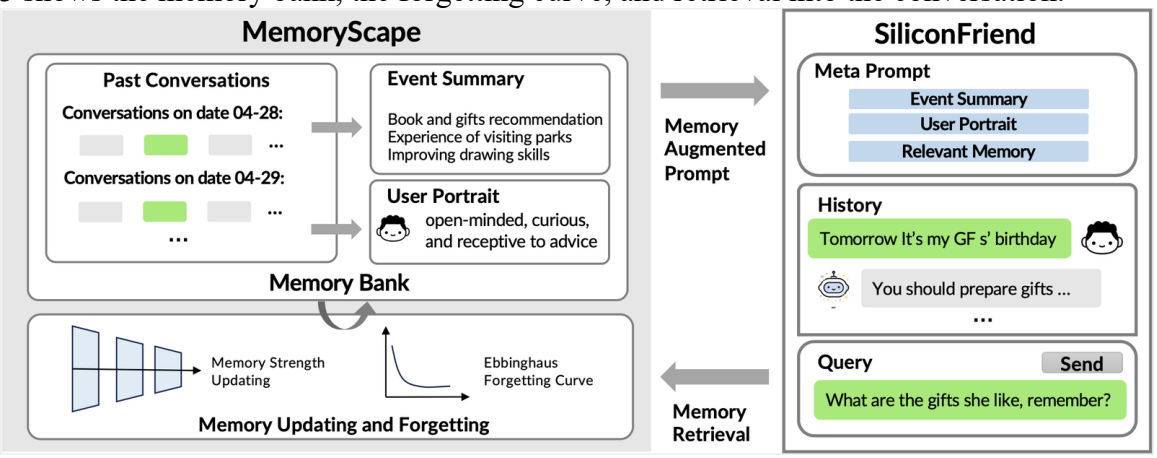


Figure 2.3: The MemoryBank framework with updating and forgetting. Source: Zhong et al. (2024).

2.3.4 HippoRAG

Gutiérrez et al. (2024) built HippoRAG on the hippocampal indexing theory, in which a compact index links the scattered traces of an experience (Teyler & DiScenna, 1986). A language model extracts entities from stored passages into a knowledge graph, and a Personalized PageRank walk spreads relevance from the question to connected passages. HippoRAG improved multi-hop question answering by up to 20% and was 10 to 30 times cheaper than iterative retrieval. A language model must build the graph, however, as in GraphRAG (Edge et al., 2024). Gutiérrez et al. (2025) also noted that graph methods fall considerably below standard retrieval on basic factual memory. Burrow therefore searches the text first and follows links that already exist between files. Figure 2.4 shows the pipeline.

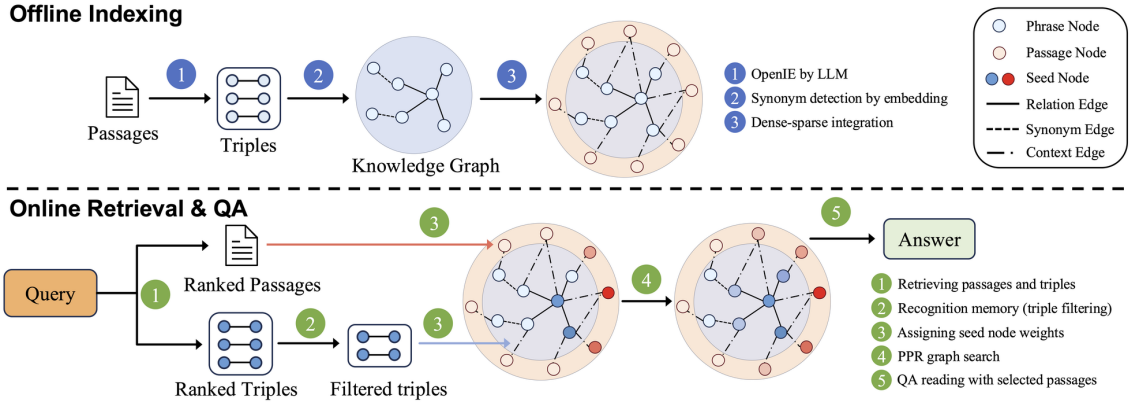


Figure 2.4: HippoRAG offline indexing and online retrieval. Source: HippoRAG project repository (Gutiérrez et al., 2024).

2.3.5 Mem0

Chhikara et al. (2025) presented Mem0, a memory layer packaged for production use. Mem0 extracts the salient facts from a conversation, consolidates them, and retrieves them at question time. On the LoCoMo benchmark of long conversations (Maharana et al., 2024), the authors reported a 26% relative improvement over OpenAI's memory on a metric judged by a language model. Mem0 is evidence that memory engineering is commercially credible. It writes memories automatically, however, where Burrow proposes a note and waits for consent. It was

evaluated on conversational recall, so it does not show how much of the gap between a small and a large model its memory closes for each type of task. Figure 2.5 shows where the memory layer sits in an agent stack.

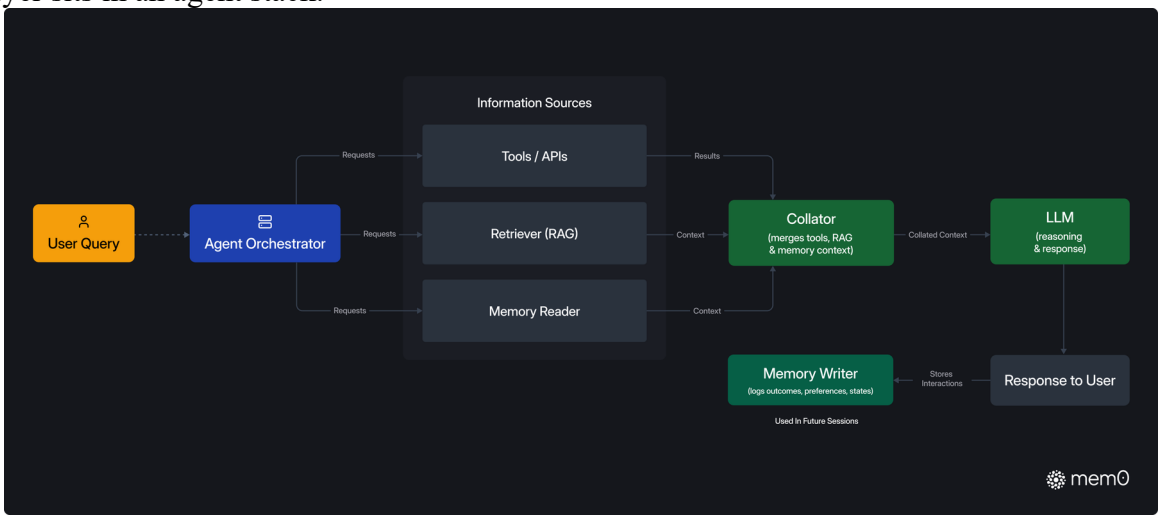


Figure 2.5: The Mem0 memory layer within an agent stack. Source: Mem0 documentation (Chhikara et al., 2025).

2.3.6 Chroma

Chroma is an open-source vector database (Chroma, 2026). Documents are cut into pieces, and each piece is stored with an identifier, an embedding vector, optional metadata, and its text. At question time the pieces nearest to the question are pasted into the prompt. It runs as an embedded library or as a distributed service. It was the baseline of this study because it is what a developer would build without a memory engine. A vector database is a store and not an engine. It answers every question with its nearest pieces, and Cuconasu et al. (2024) showed that highly ranked pieces without the answer are the ones that hurt. Chroma's own researchers reported that reliability falls as input grows (Hong et al., 2025). Figure 2.6 shows the distributed architecture.

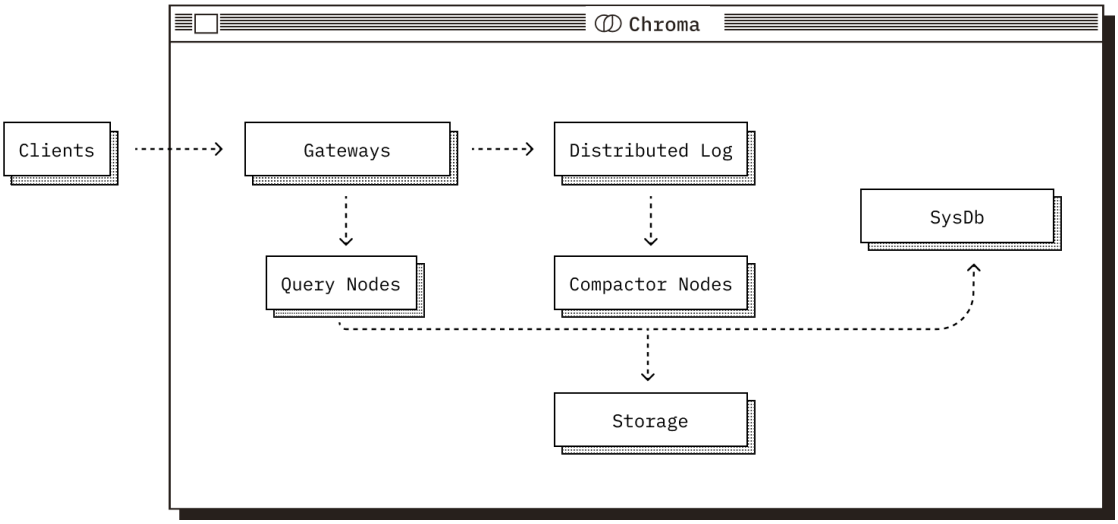


Figure 2.6: The architecture of distributed Chroma. Source: Chroma (2026), https://docs.trychroma.com.

2.3.7 Obsidian

Obsidian is a note-taking application. Its vault is an ordinary folder of plain Markdown files on the person's own disk, and the application adds links between notes without locking them into a database (Obsidian, 2025a). Its graph view draws each note as a circle and each link as a line (Obsidian, 2025b). A-Mem applied the same idea to agents by having a model build a network of linked notes (Xu et al., 2025). Burrow instead starts from the links a person already wrote, in a vault that Obsidian can open. Obsidian is a tool for a person to write and read. It does not select what a language model should read, fit that selection to the model, or measure what the model gains. Figure 2.7 shows the graph view.

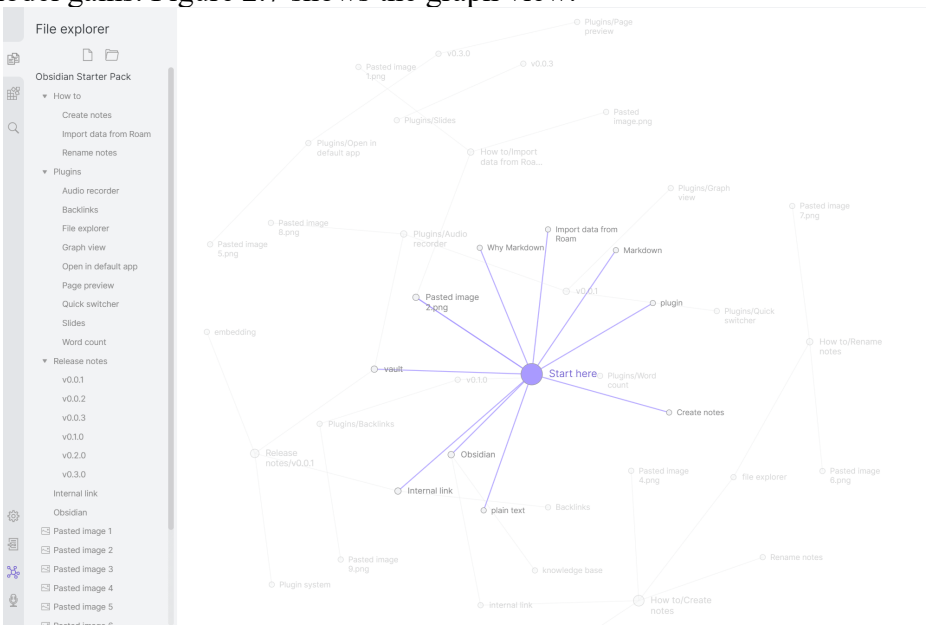


Figure 2.7: The graph view of an Obsidian vault. Source: Obsidian (2025b), https://help.obsidian.md/plugins/graph.

2.3.8 Claude Code and the Model Context Protocol

Claude Code is a coding assistant from Anthropic that reads a codebase, edits files, and runs commands for the developer (Anthropic, 2026a). It keeps a project memory in plain files (Anthropic, 2026b), and it loads written skills in stages so that an unused skill costs little context (Anthropic, n.d.). It reaches outside tools and data through the Model Context Protocol, an open standard between AI applications and data sources (Model Context Protocol, 2026). Hou et al. (2026) catalogued 16 threat scenarios for servers of this protocol. Claude Code is the strongest form of the cloud side of this study. Its model runs in the cloud, so questions and files leave the device. Burrow was built to work beside it through its own protocol server. Figure 2.8 shows the protocol.

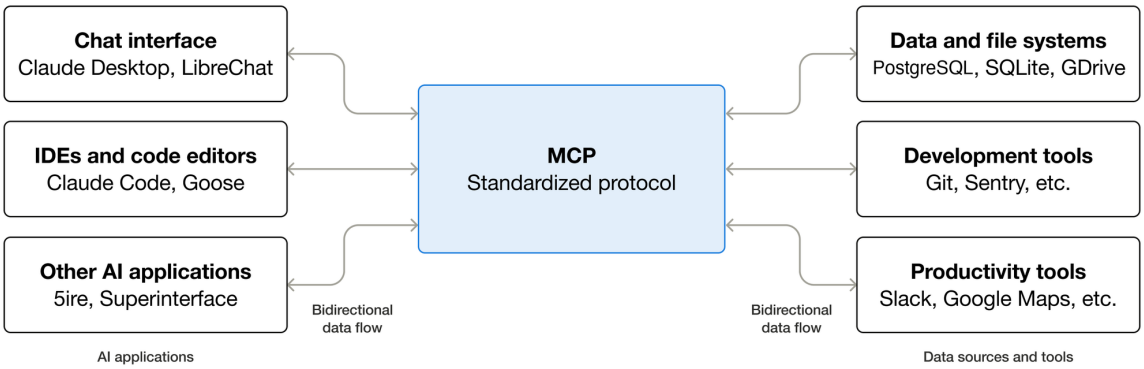


Figure 2.8: The Model Context Protocol between AI applications and data sources. Source: Model Context Protocol (2026), https://modelcontextprotocol.io.

Table 2.1 compares the eight systems by their core idea, their strength, and what each leaves open for the on-device setting.

Table 2.1: Comparison of the related works

System	Core idea	Strength	Limitation for the on-device setting
MemGPT (Packer et al., 2023)	Pages context in and out of the window like an operating system	Sustains very long interactions on a fixed window	Manages context length; does not target capability gain or token cost
Generative Agents (Park et al., 2023)	Memory stream scored by relevance, recency, and importance	Simple, effective retrieval scoring	Designed for simulation; verbose natural-language records
MemoryBank (Zhong et al., 2024)	Memories strengthen and decay over time	Principled forgetting	No accounting of token cost or device latency
HippoRAG (Gutiérrez et al., 2024)	Knowledge-graph index walked with Personalized PageRank	Strong multi-hop recall	A language model must build the graph, which is heavy for a device
Mem0 (Chhikara et al., 2025)	Salient facts extracted and consolidated into a managed store	Production-ready; gains reported on a long-conversation benchmark	Writes memories without consent; gap closing per task type not measured
Chroma (Chroma, 2026)	Vector database queried for the pieces nearest to the question	Simple, runs locally, the standard pipeline	A store, not an engine; same retrieval for every question and every model
Obsidian (Obsidian, 2025a)	Vault of plain Markdown notes joined by links	The person owns the files; structure is visible	Made for a person to read; does not supply memory to a model
Claude Code (Anthropic, 2026a)	Cloud coding assistant with project memory, extended through the Model Context Protocol	Large model with a memory of the project	Model runs in the cloud; data leaves the device

2.4 Gaps in Related Works

The reviewed work establishes each ingredient separately. Retrieval supplies the long-tail knowledge that small models lack (Kandpal et al., 2023; Mallen et al., 2023). Memory systems store, link, and forget information across long interactions (Chhikara et al., 2025; Packer et al., 2023; Zhong et al., 2024). Methods that decide when to retrieve exist, but Self-RAG needs a specially trained model (Asai et al., 2024) and Adaptive-RAG needs a trained classifier (Jeong

et al., 2024). Moskvoretskii et al. (2025) compared 35 such methods and found that simple uncertainty estimates often matched complex pipelines in answer quality.
What is missing is the measurement on a device. Narayan et al. (2025) came nearest. Their on-device model recovered 87% of a cloud model's performance, and 97.9% when the cloud model split the task into small local jobs, but it depended on the cloud throughout. No reviewed work places a memory engine beside a frozen small model on a consumer laptop and reports what fraction of the gap to a cloud model it closes. None reports this separately for knowledge-bound and reasoning-bound tasks, with token and latency accounting, against both a plain vector database and a cloud model. This study addressed that gap.

2.5 Conceptual Framework

The framework that structured the study has four parts. The independent variable is the configuration of the memory engine: its Decide, Fit, Check, and Learn steps and the skills it hands to the model with the question. The process is question answering on the device, where the frozen on-device model answers from the question and the memory assembled within the token budget. The dependent variables are the fraction of the capability gap closed, the accuracy gained per token of memory, and the latency on the target device. Task type, knowledge-bound or reasoning-bound, moderates the relationship. The model's weights were held constant, so every measured gain is attributable to the engine. Figure 2.9 presents the framework.

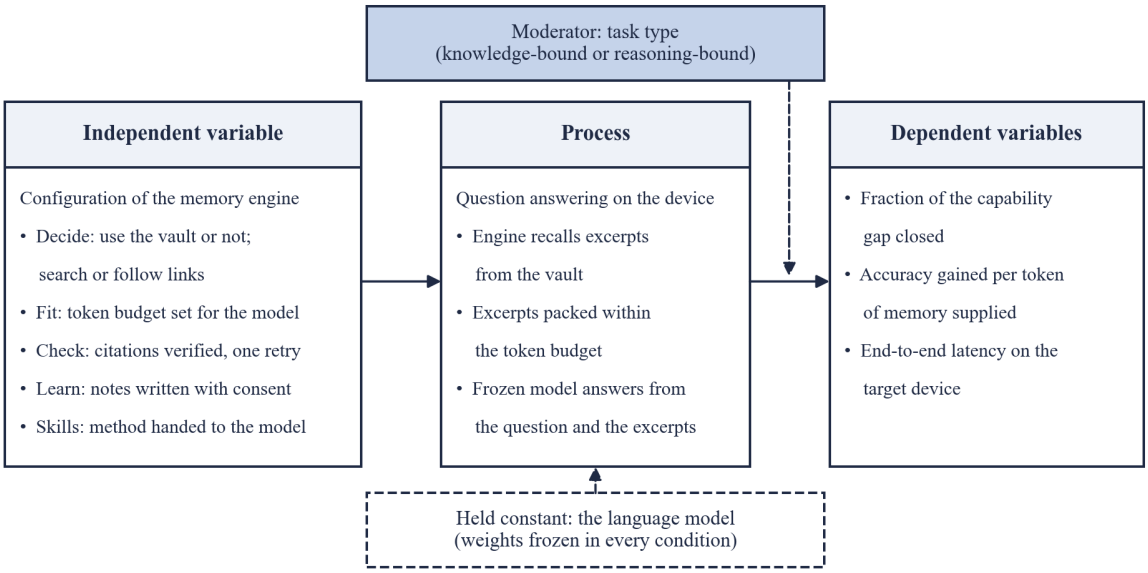


Figure 2.9: Conceptual framework of the study.

Chapter 3: Methodology

3.1 Introduction

This chapter describes how the study was carried out. It states the research paradigm and the four stages of the experiment. It then gives the development methodology and the reason it was chosen. The analysis and design followed Object-Oriented Analysis and Design, and the chapter names each diagram that Chapter 4 presents. It closes with the deliverables of the study and the tools that were used.

3.2 Research Paradigm

The study adopted an experimental research paradigm within a Design Science Research frame. Design science applies because the study built a new artefact, the Burrow memory engine, and an artefact of this kind must be evaluated against stated measures of usefulness (Hevner et al., 2004). The work followed the stages of Peffers et al. (2007): identify the problem, define the objectives, design and develop, demonstrate, evaluate, and communicate. The evaluation was a controlled experiment. The same questions were put to an on-device model with and without the engine, and to a large cloud model, and the answers were scored against keys. The four subsections below describe the data, its processing, the treatment of the models, and the testing.

3.2.1 Data Acquisition

Two kinds of data were acquired, so that questions bound by knowledge could be separated from questions bound by reasoning.

The knowledge data was a curated vault on three subjects that an on-device model cannot know well. The first subject was Kenyan data protection law: the Data Protection Act (2019) and its 2021 regulations, taken from Kenya Law. The second was software documentation dated 2026, published after the on-device models were trained: the Model Context Protocol specification, the release notes of Ollama, and the release notes of Python 3.15. The third was a private software project: the source code and the decision record of Burrow itself.

The reasoning data was 50 problems from GSM8K, a public benchmark of grade-school mathematics problems (Cobbe et al., 2021). The problems were drawn once from the test split with a fixed random seed.

3.2.2 Data Processing

The documents of the three subjects were converted to Markdown files and placed in one vault. The engine then indexed the vault. Each file was cut into pieces. Each piece was entered in a word index and given an embedding vector, and the links between files were recorded. The three subjects shared one vault, so the documents of two subjects stood as distractors for the third.

For each subject, 25 single-fact questions and 15 multi-fact questions were written from the documents. Each question carried an answer key and the passage that supports it. A script checked every key against the text of the vault. The vault was built from the source documents only and never from the questions or their keys. The same questions were used in every set-up, so every comparison was paired.

3.2.3 Model Training

No model was trained or fine-tuned in this study. The on-device models and the cloud model were used exactly as their publishers released them, and their weights stayed frozen in every

set-up. This is the premise of the study: the engine works beside the model and does not change it.
What was set in place of training were the thresholds of the engine. These are the closeness below which the vault is left out of an answer, and the closeness at which a skill is matched to a question. They were set on development questions written for that purpose: 11 questions for the vault threshold and 26 labelled questions for the skill thresholds. The development questions were kept apart from the scored questions, and the scored questions were not read while the engine was tuned.

3.2.4 Model Validation and Testing

Six set-ups were tested on the same questions. They were the on-device model alone, the on-device model with plain retrieval, the on-device model with a vector database baseline built on Chroma, the on-device model with the engine, the cloud model alone, and the cloud model with the memory prepared by the engine. The Chroma baseline used the same vault, the same embedding model, and the same limit of 12,000 characters as plain retrieval. Each on-device set-up was run with the main model and with the small model. Every model was run at a temperature of zero.
An answer was scored by its answer key. A knowledge answer was right when it contained every required fact of its key. A reasoning answer was right when its last number equalled the key. Accuracy was the share of right answers in a set-up. The central measure of the study, the fraction of the capability gap closed, was computed for each task type as shown in Equation 3.1, where S is the accuracy of a set-up. The floor is the on-device model alone, the ceiling is the cloud model alone, and the treatment is the on-device model with the engine.

$$Gap\ closed\ = \frac{S_{\text{treatment}} - S_{\text{floor}}}{S_{\text{ceiling}} - S_{\text{floor}}}$$

Equation 3.1: Fraction of the capability gap closed.

A value of zero means the engine added nothing. A value of one means the on-device model with the engine matched the cloud model. Where the cloud model was not ahead of the on-device model alone, the measure is not defined and was reported as such.
Cost was measured beside accuracy. Every answer logged the tokens read, the tokens written, the seconds spent before the model was called, and the seconds the model took. Paired differences between set-ups were tested with an exact two-sided sign test, because each answer is simply right or wrong. The engine was frozen as one binary before the first scored run, and the evaluation plan was written down before that run. Faults found after the freeze were reported and were not fixed.

3.3 System Development Methodology

The system was developed by incremental prototyping. The work was divided into five increments. Each increment went through analysis, design, build, and evaluation, and each ended with a working version that was measured before the next began. The engine was first prototyped under the working name CAMR in Python and was rebuilt as Burrow in Swift after the Sprint II review of 5 November 2026.

3.3.1 Justification

Incremental prototyping suited the study for three reasons. First, the evaluation harness had to exist before the engine, because every part of the engine was judged by what the harness measured. Building it first removed the largest risk of the project, which was reaching the end with no way to produce results. Second, the engine is a chain of separate steps. Adding one

step at a time made it possible to measure what each step contributed. Third, the requirements changed during the project. After the review of 5 November, the surfaces through which a person uses the engine were added, and an incremental plan could take them in as one more increment without discarding the measured work.

3.4 Methodology Diagram

Figure 3.1 shows the five increments in order. Each box repeats the same four activities, and the label above each box names what that increment delivered.

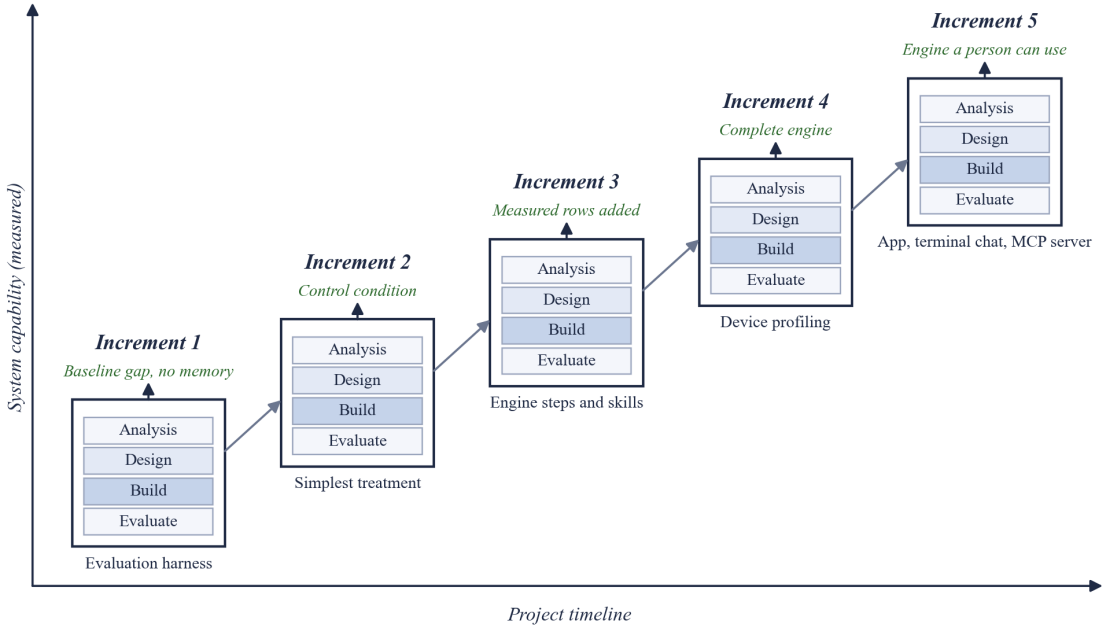


Figure 3.1: Incremental development of the Burrow engine.

3.4.1 Increment 1: Evaluation Harness

The harness was built first. It loads the questions, runs the on-device and cloud models, scores the answers against their keys, and logs tokens and seconds. It was validated by measuring the gap between the two models with no memory at all.

3.4.2 Increment 2: Simplest Treatment

The second increment added the simplest complete memory. Files were indexed with embeddings, pieces were retrieved by search alone, and they were packed into a fixed budget. This version became the control against which the engine was compared.

3.4.3 Increment 3: Engine Steps and Skills

The third increment added the parts of the engine one at a time. These were the walk over links between files, the Decide, Fit, Check, and Learn steps, and the skills. Each addition was measured against the control.

3.4.4 Increment 4: Device Profiling

The complete engine was profiled on the target laptop. The tokens and the seconds of every answer were measured, with the time spent before the model was called reported separately.

3.4.5 Increment 5: Application, Terminal Chat, and MCP Server

The last increment added the surfaces through which the engine is used. These were a native Mac application, a terminal chat, and a Model Context Protocol (MCP) server that opens the same memory to outside models.

3.5 System Analysis Diagrams

The analysis followed Object-Oriented Analysis and Design, because the engine was written as a set of types that hold their own data and behaviour. The diagrams were drawn in the Unified Modeling Language (UML). Four analysis diagrams were chosen, and Chapter 4 presents them.

3.5.1 Use Case Diagram

The use case diagram shows the boundary of the system, its actors, and what each actor uses the engine for. It was chosen because the engine serves three kinds of actor: a person, a local model, and an outside model or tool. One use case, "Ask a question", is also described in a table.

3.5.2 Sequence Diagram

The sequence diagram shows the messages exchanged, in order, when a question is answered. It was chosen because the order of the five steps is the design of the engine. A second sequence diagram shows an outside model using the engine through MCP.

3.5.3 Activity Diagram

The activity diagram shows the flow of work from a question to a kept note, with its decision points. Swimlanes separate what the person, the engine, and the local model each do. It was chosen to make clear which party makes each choice.

3.5.4 Class Diagram

The class diagram shows the main types of the engine, their operations, and the relationships between them. It was chosen because it maps directly to the Swift code and guided how the library was divided.

3.6 System Design Diagrams

Three kinds of design diagram were chosen to show how the system was put together.

3.6.1 Database Schema

The database schema shows the tables of the index and how they relate. It was chosen because the index is one SQLite file, and its tables decide what can be searched by words, by meaning, and by links.

3.6.2 Wireframes

The wireframes show the layout of the screens before they were built. They cover the main window of the Mac application, the note editor, the step where the person agrees to what is

kept, and the terminal chat. They were chosen to plan where the engine's decisions are shown to the person.

3.6.3 System Architecture

The system architecture diagram shows the components of the system and their dependencies. It was chosen to show that all memory logic sits in one core library that every surface uses. A diagram of the engine's five steps and a deployment diagram accompany it. The deployment diagram shows which parts run on the laptop and which connections leave it.

3.7 Deliverables

The study produced five deliverables.

3.7.1 The Burrow Engine Library and Command-Line Tool

A Swift package that implements the vault, the index, and the five steps of the engine, with a command-line tool named `burrow`. Both run entirely on the target laptop.

3.7.2 The Mac Application and Terminal Chat

A native Mac application in which a person writes notes, follows their links, and asks questions of the vault. A terminal chat offers the same engine from the command line.

3.7.3 The MCP Server

A server that gives outside models and tools the same memory through MCP. Its tool `vault_prepare` and the command `burrow brief` return the engine's decision, a skill, and the excerpts for a task.

3.7.4 The Evaluation Harness

A set of Python scripts that run every set-up, score the answers, and write the tables and charts from the logged results. Every experiment in the study can be repeated from the command line.

3.7.5 Results and Documentation

The logged results of every run, this report, and the instructions, configuration files, and prompt templates needed to reproduce the results.

3.8 Tools and Techniques

3.8.1 Swift

The engine, the command-line tool, the terminal chat, the MCP server, and the Mac application were written in Swift. Swift is the native language of the target platform, so the engine runs as one compiled program with no interpreter.

3.8.2 SQLite with FTS5

The index is one SQLite database file, which needs no server process. The word index uses the FTS5 full-text extension of SQLite (SQLite, 2026). The embedding vectors are stored in the same file.

3.8.3 Ollama and the Local Models

The local models were served by Ollama, which runs quantised open-weight models on consumer hardware (Ollama, 2026). The main model was Gemma 4 12B (Gemma Team, 2026). The small model was Qwen3.5 4B (Qwen Team, 2026).

3.8.4 The Hosted API

The cloud reference model, Claude, was reached only through its hosted API. It was pinned to one version for the whole study, so that the ceiling stayed the same across runs.

3.8.5 The Embedding Model

Embeddings were produced on the laptop by the qwen3-embedding model at 0.6 billion parameters, served through Ollama (Zhang et al., 2025). The engine and the baseline used the same model.

3.8.6 Chroma

Chroma is an open-source vector database (Chroma, 2026). It was run locally to build the vector database baseline of section 3.2.4.

3.8.7 LangChain

LangChain is a framework for building applications around language models (LangChain, 2026). An adapter exposes Burrow as a LangChain retriever, so the engine can take the place of a vector database in an existing pipeline.

3.8.8 Python

The evaluation harness was written in Python. Python was also used for the scripts that check the answer keys and draw the charts.

3.8.9 Git and GitHub

Git was used for version control, and the repository was hosted on GitHub. The state of the code at each evaluation was marked with a tag, so that a result can be traced to the code that produced it.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the analysis and design of Burrow, the memory engine of this study. It gives the functional and non-functional requirements, and then the diagrams named in Chapter 3. The work followed Object-Oriented Analysis and Design, and the diagrams use the Unified Modeling Language (UML). One idea runs through the chapter: the engine takes five steps around a frozen local model, which are Decide, Fit, Answer, Check, and Learn.

4.2 System Requirements

The requirements were worked backwards from the objectives in section 1.3 and from the three mechanisms of the proposal: a write policy, a retrieval policy, and a token budgeting mechanism. The test cases of Chapter 5 refer to the identifiers given here.

4.2.1 Functional Requirements

Table 4.1 lists the ten functional requirements. FR-01 to FR-03 cover the memory, FR-04 to FR-09 are the five steps of the engine, and FR-10 opens the engine to outside models.

Table 4.1: Functional requirements

ID	Requirement
FR-01	The engine shall index a vault of Markdown notes, documents, and linked code folders on the device, and later re-read only the files that changed.
FR-02	The engine shall rank the pieces of the vault by word match and by meaning, and return excerpts with their file and line numbers within a character budget.
FR-03	The engine shall follow the links between files for up to two hops from the best matches.
FR-04	Decide: the engine shall judge from one search whether a question needs the vault, and whether to search or to follow links, and state the reason in words.
FR-05	Decide: the engine shall hand the model at most one skill, chosen by how closely it fits the question or else by the kind of material found.
FR-06	Fit: the engine shall measure a local model once and set from that measurement how many characters the model is shown.
FR-07	Answer: the engine shall have a local model answer from the excerpts, naming a file and line for each statement.
FR-08	Check: the engine shall check every cited file and line against the excerpts shown, ask once more if a citation was not shown, and report any left unverified.
FR-09	Learn: the engine shall propose at most three notes from a conversation, each backed by the person's own words, and write a note only when the person accepts it.
FR-10	The engine shall give the same decision, skill, and excerpts to an outside model through a command and through a Model Context Protocol (MCP) tool, without calling a chat model itself.

4.2.2 Non-Functional Requirements

Table 4.2 lists the six non-functional requirements. No limit on response time was set in advance, so NFR-04 requires that cost be measured on every answer.

Table 4.2: Non-functional requirements

ID	Requirement
NFR-01	Privacy: the engine shall send notes and code only to the Ollama service on the same machine, at the loopback address 127.0.0.1.
NFR-02	Durability: the person's files shall be the only lasting record, and the index shall be rebuilt from them if it is deleted.
NFR-03	Safety: text from the vault shall be given to a model as material to read, with a rule that no instruction written inside it is followed.
NFR-04	Cost: for every answer the engine shall report the tokens read, the tokens written, the seconds before the model was called, and the seconds the model took.
NFR-05	Footprint: the index shall be one embedded database file inside the vault's `.burrow` folder, with no database server process.
NFR-06	Portability: the engine shall build as one Swift package with no third-party dependency, and its tests shall run with one command.

4.3 System Analysis Diagrams

4.3.1 Use Case Diagram

Figure 4.1 shows the boundary of the system and its three kinds of actor: the person who owns the vault, the local model served by Ollama, and an outside model or tool that brings its own model. Asking a question and preparing a task for another model both include the same use case, deciding and packing the excerpts. Only the path of the local model includes checking the citations, because only there does the engine see the answer.

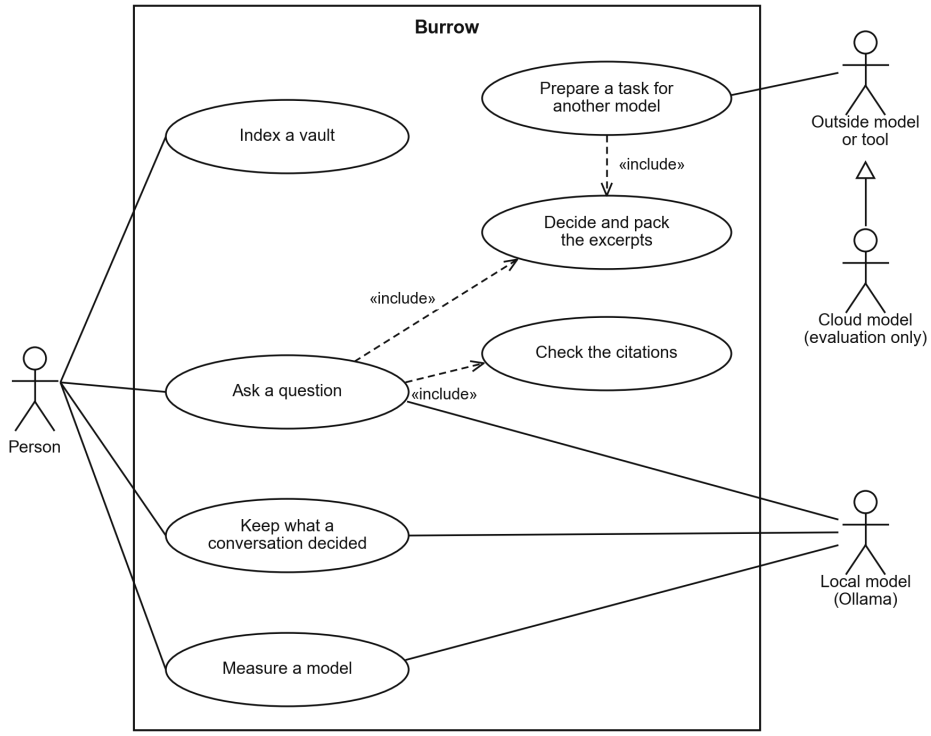


Figure 4.1: Use case diagram of Burrow.

4.3.2 Use Case Description

Table 4.3 describes the use case that carries the study, "Ask a question". It exercises four of the five steps: Decide, Fit, Answer, and Check.

Table 4.3: Use case description: Ask a question

Item	Description
Actors	Person (primary), local model (supporting)
Precondition	The vault has been indexed. Ollama is running with a chat model.
Main flow	1. The person asks a question. 2. The engine searches once and decides: leave the vault out, search, or follow links. 3. The engine picks a skill and packs the excerpts within the model's budget. 4. The local model answers. 5. The engine checks each file and line cited. 6. The person receives the answer, the decision line, and the sources.
Alternative flows	At step 2 nothing in the vault is close, so the model answers without excerpts. At step 5 a citation was not among the excerpts, so the model is asked once more and the better grounded answer is kept.
Exceptions	Ollama cannot embed the question, so ranking continues on words alone. Ollama cannot answer, so the failure is reported and no answer is shown.
Postcondition	No file in the vault has changed. Any citation that could not be verified is named.

4.3.3 Sequence Diagram

Figure 4.2 shows the messages exchanged when a person asks a question through the engine. The model profile is fetched first, so the budget is known before any search begins. The local model is called once in the usual case and twice at most, the second time with the wrongly cited places named.

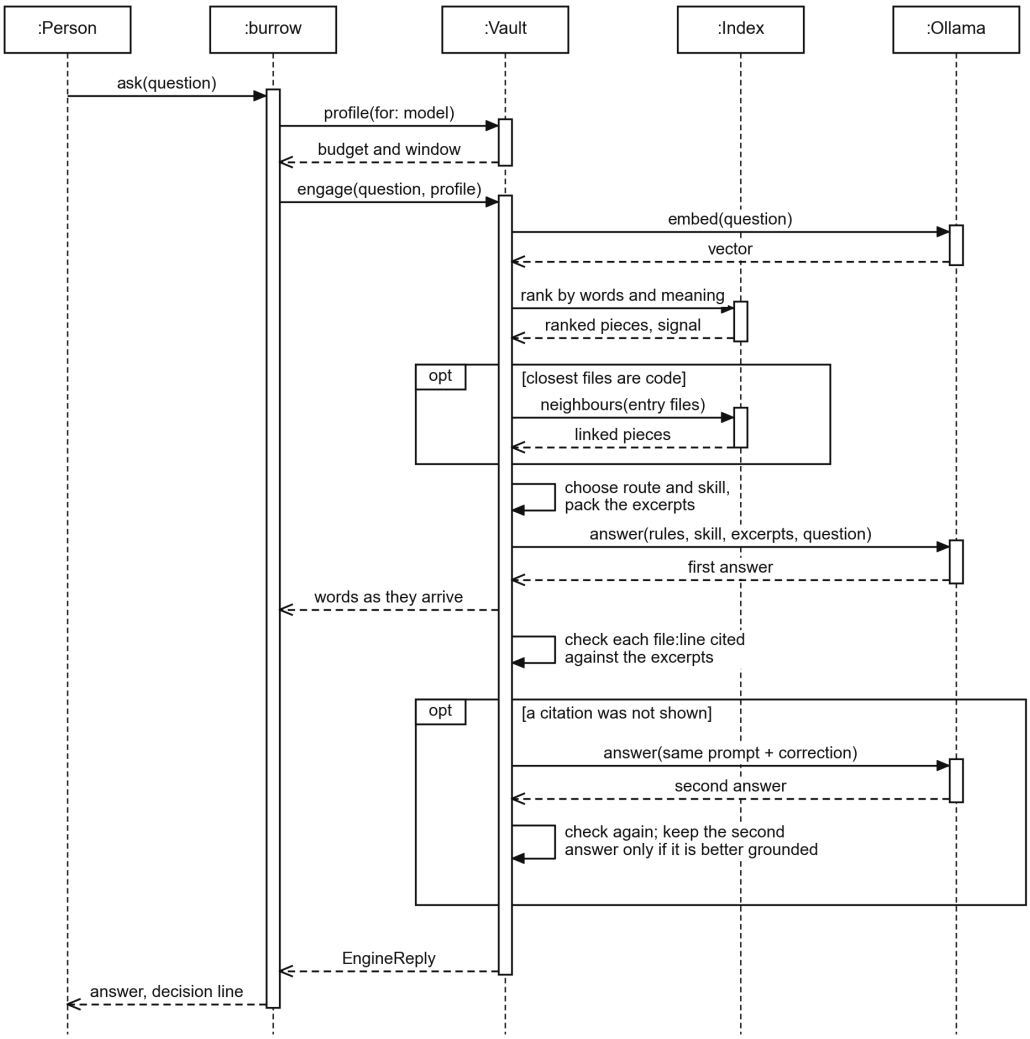


Figure 4.2: Sequence diagram of a question asked through the engine.

Figure 4.3 shows an outside model using Burrow through the MCP tool `vault_prepare`. The only model Burrow calls is the embedding model. The outside model receives the decision, a skill, and the excerpts, and writes the answer itself.

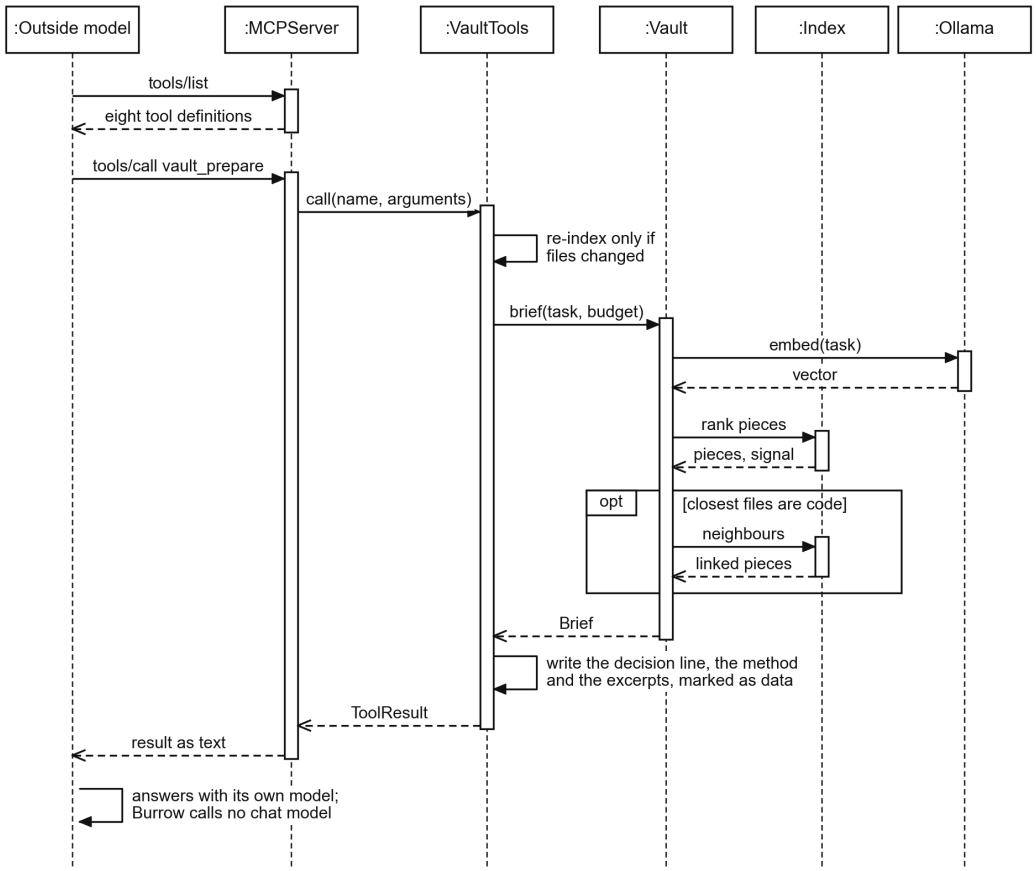


Figure 4.3: Sequence diagram of an outside model preparing a task through MCP.

4.3.4 Activity Diagram

Figure 4.4 follows the engine's loop from a question to a kept note, in three swimlanes for the person, the engine, and the local model. Every choice about what the model reads is made in the engine lane, before the model is called. Each time the model returns something, the engine examines it before it is used.

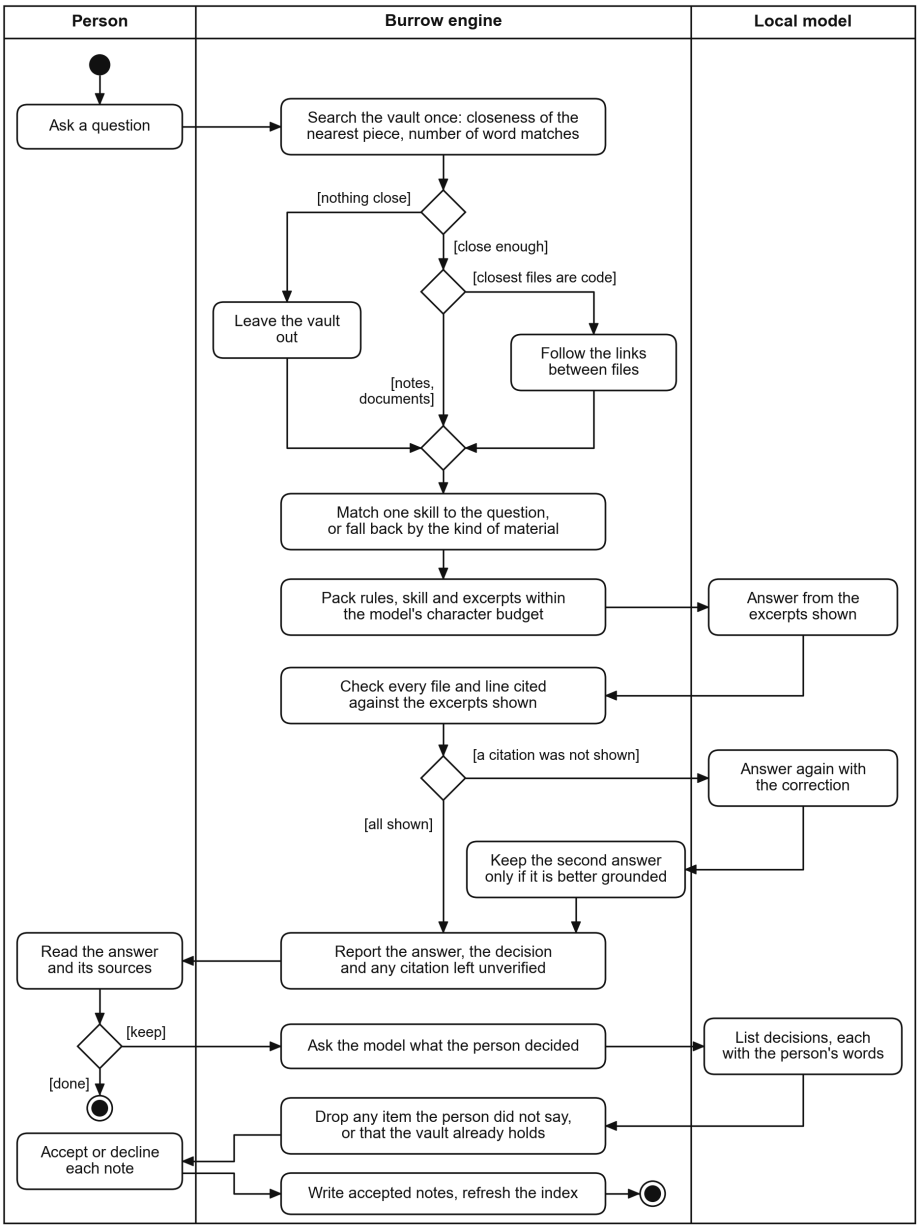


Figure 4.4: Activity diagram of the engine's loop.

4.3.5 Class Diagram

Figure 4.5 shows the main types of the engine as they were built. `Vault` is the single entry point: `brief` decides and packs, `engage` adds the answer and the check, and `lessons` and `keep` are the two halves of Learn. A `Brief` contains no answer, which is why it can be handed to an outside model. `Embedder` and `Answerer` are protocols, so the tests run the whole loop with a scripted model.

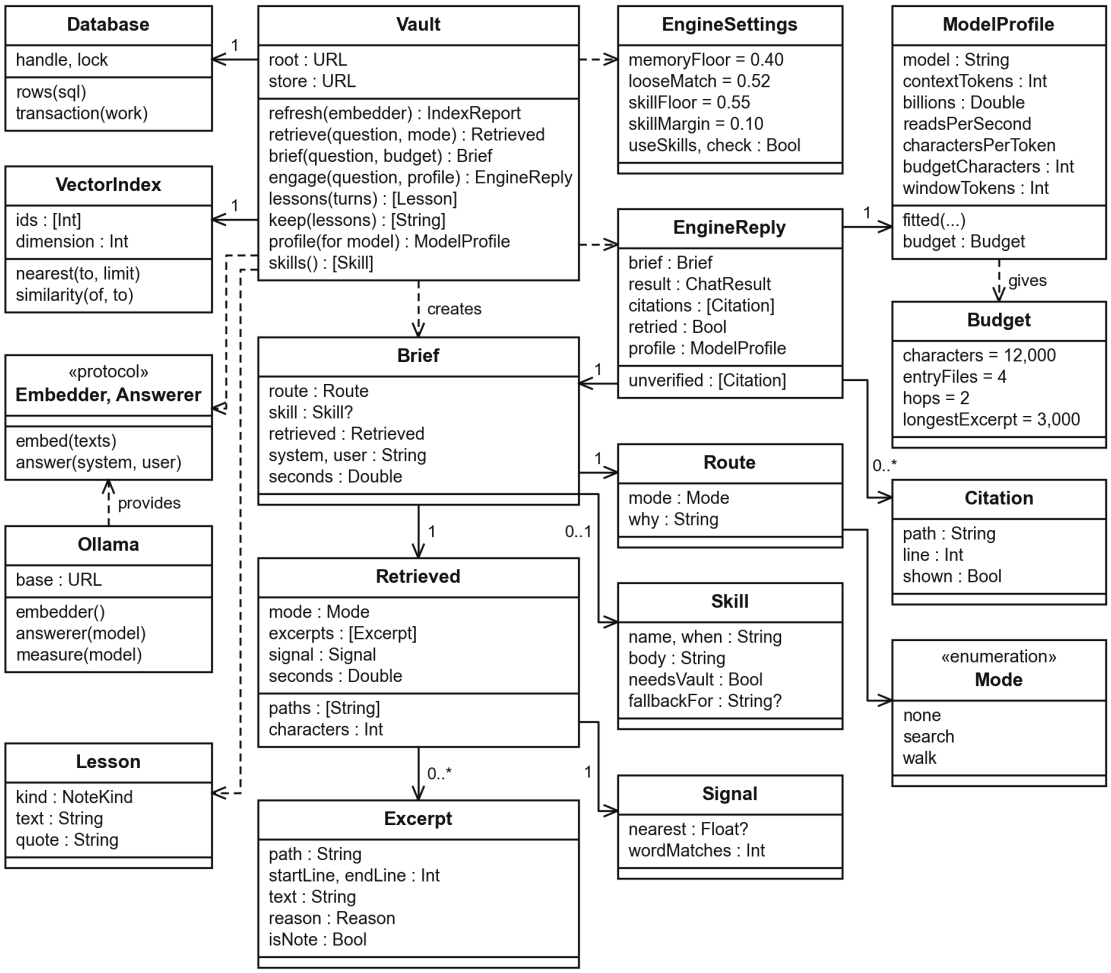


Figure 4.5: Class diagram of the engine.

4.4 System Design Diagrams

4.4.1 System Architecture

Figure 4.6 shows the components of Burrow in three layers: the surfaces that a person or a tool touches, a shared set of tools, and the core. All memory logic is in BurrowCore, and every surface depends on it. The Ollama client is the only part that leaves the process.

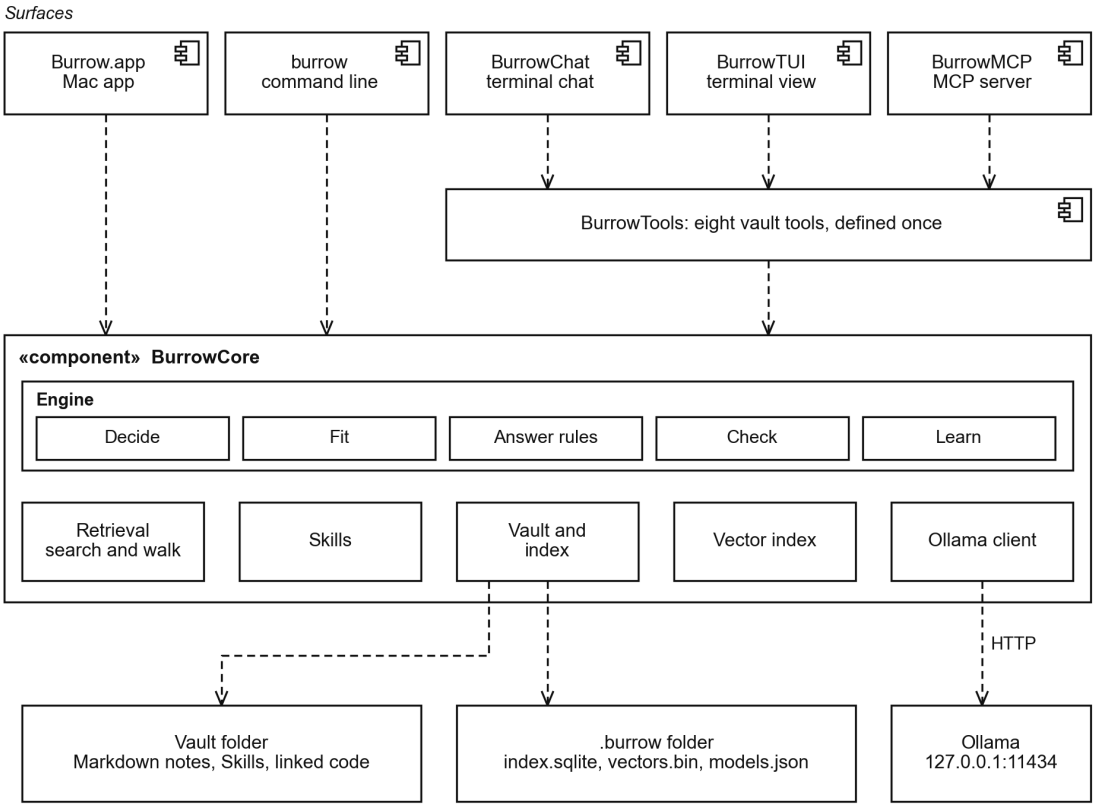


Figure 4.6: Architecture of Burrow.

Figure 4.7 shows the five steps of the engine and what each reads and writes. The steps map to the mechanisms of the proposal. Learn is the write policy, Decide and the retrieval behind it are the retrieval policy, and Fit is the token budgeting mechanism. Check was added in this study. The recency and importance scores of the proposal were not built, and ranking is by words, meaning, and links.

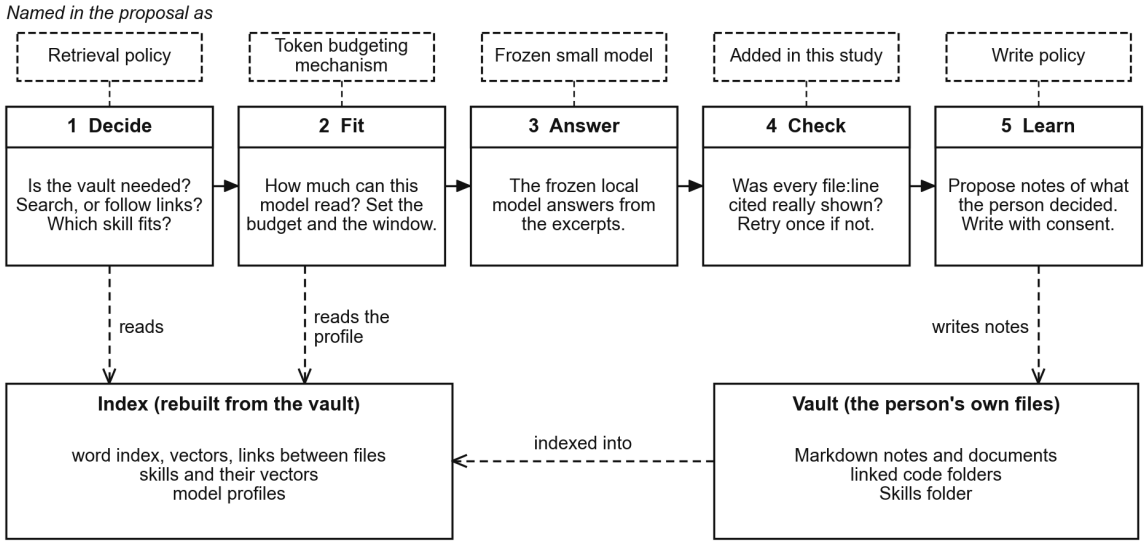


Figure 4.7: The five steps of the engine and the mechanisms of the proposal.

Retrieval ranks each piece twice, by word match with BM25 (Robertson & Zaragoza, 2009) and by meaning. Meaning is the cosine between the vector of the question, q, and the vector of a piece, v, as in Equation 4.1.

$$cos(q, v) = \frac{q \cdot v}{\|q\| \, \|v\|}$$

Equation 4.1: Cosine similarity between a question and a piece.

The two rankings are merged by reciprocal rank fusion (Cormack et al., 2009), as in Equation 4.2, where the rank of piece p in ranking r counts from 1 and w(p) lowers the weight of test files and of the output of past runs.

$$score(p) = w(p) \sum_{r \in \{words, meaning\}} \frac{1}{60 + rank_r(p)}$$

Equation 4.2: Merged score of a piece.

When links are followed, a neighbouring file n is ranked by Equation 4.3, where b(n) is its word match scaled so that the best file scores 1, h is the hop number, and k(n) depends on the kind of link.

$$s(n) = w(n) \left(\frac{b(n)}{h} + k(n) \right)$$

Equation 4.3: Score of a neighbouring file during a walk.

4.4.2 Database Schema

Figure 4.8 shows the logical schema of the index, which is one SQLite file. A file is cut into pieces stored as `chunks`, and a piece has at most one vector, because the index also works on words alone. `chunks_fts` is the full-text table, and `links` holds the links between files. Nothing in the schema is a source of truth: the whole index can be rebuilt from the Markdown files.

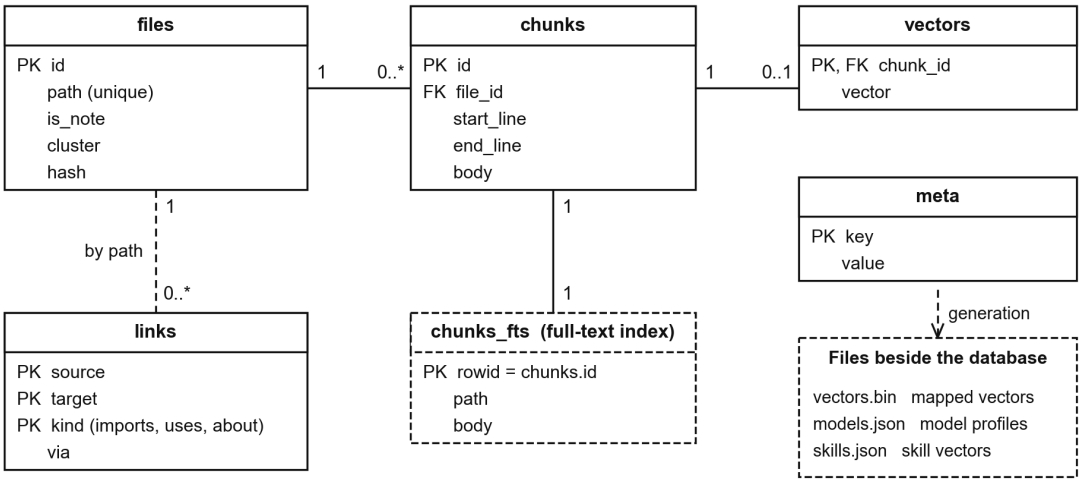


Figure 4.8: Logical schema of the index.

4.4.3 Wireframes

Figure 4.9 is the main window of the Mac application with the Ask panel open. The decision line sits directly under the answer and says whether the vault was read, why, and which skill was handed to the model. A warning line appears only when the answer cites a place the model was not shown.

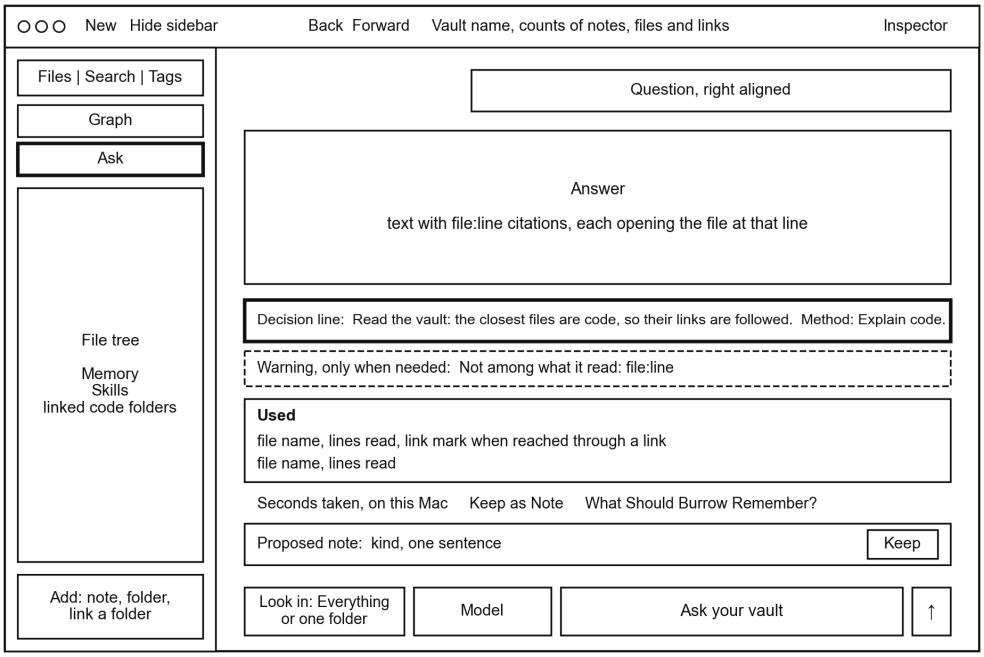


Figure 4.9: Wireframe of the main window with the Ask panel.

Figure 4.10 is the note editor. A note is a plain Markdown file, and a link to another note is written between double square brackets, with the matching names offered as it is typed. The panel on the right shows the note's neighbours as a small graph and lists the notes that link to it and the files it links to.

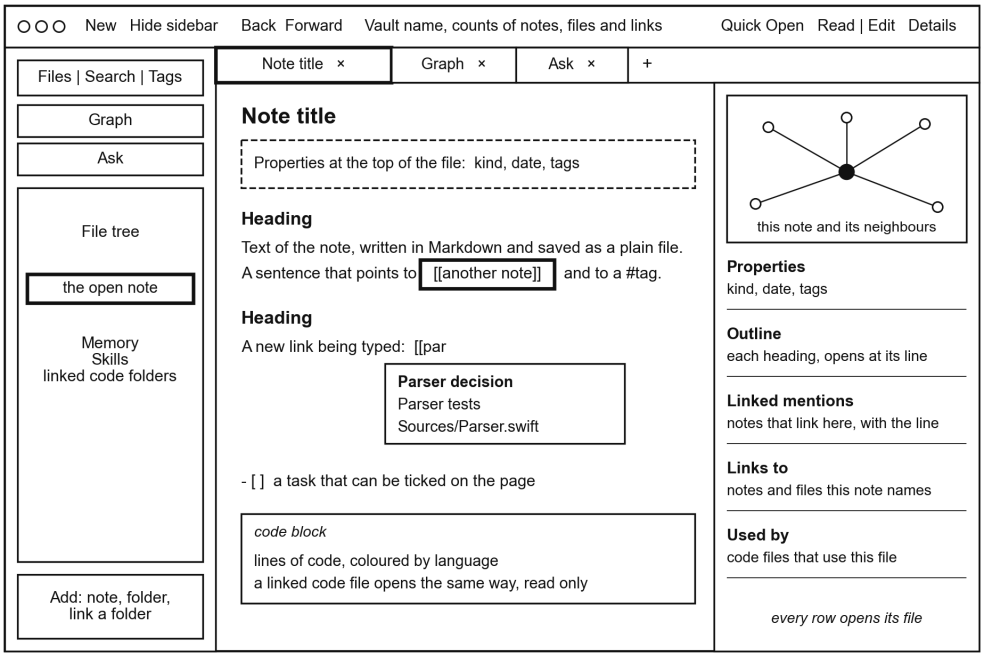


Figure 4.10: Wireframe of the note editor with its links.

Figure 4.11 is the step where the person agrees to what is kept. The engine offers at most three notes, each with its kind and one sentence. Each note has its own Keep button, so nothing is written to the vault without an act of the person. A kept note appears as a new file in the Memory folder.

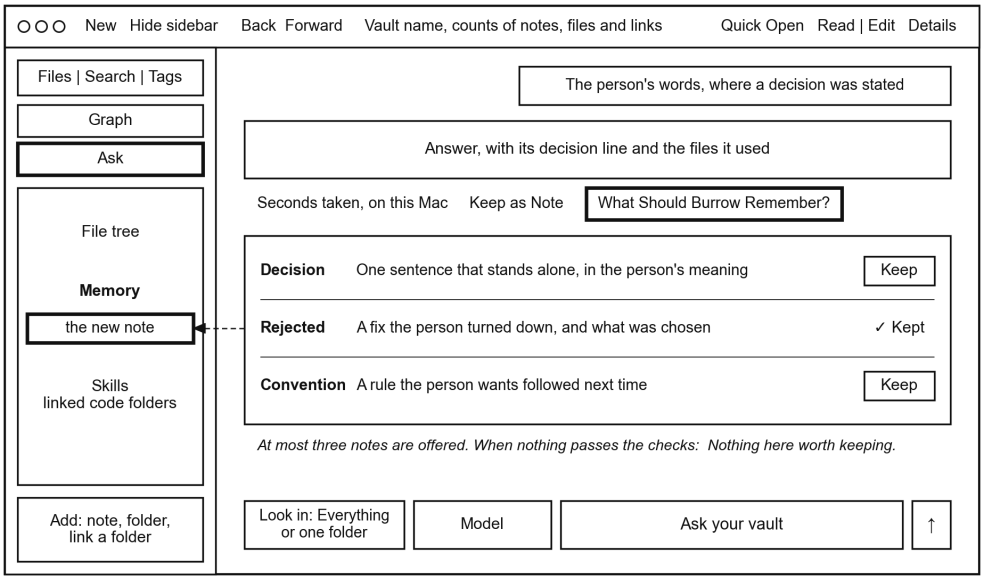


Figure 4.11: Wireframe of the step where the person keeps what was decided.

Figure 4.12 is the terminal chat, started with `burrow chat`. It shows the same two moments as the window. The line under each question is the engine's decision, and the `/keep` command offers each note in turn and waits for a yes before it writes.

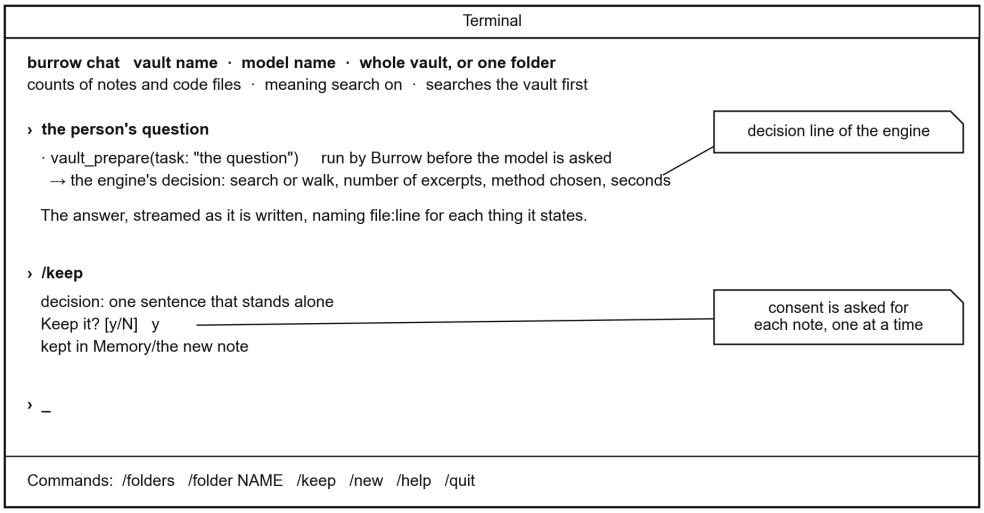


Figure 4.12: Wireframe of the terminal chat.

4.4.4 Deployment Diagram

Figure 4.13 shows where each part runs. The application, the command-line tool, the vault, the index, and both local models are on one laptop, and the engine reaches Ollama only on the loopback address. Two connections leave the laptop, and neither is made by the engine: an MCP client that the person chooses to connect, and the evaluation harness calling the cloud model.

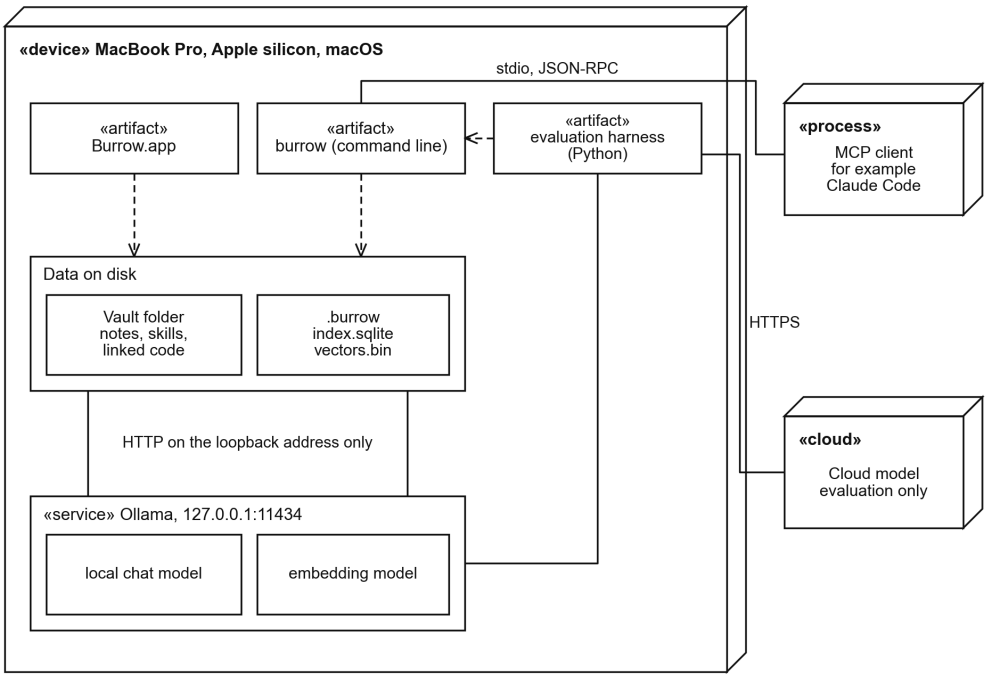


Figure 4.13: Deployment diagram of Burrow.

Chapter 5: System Implementation and Testing

5.1 Introduction

This chapter presents the engine as it was built and what the experiments measured. It states the environment, describes the dataset, shows each part of the engine at work, reports the experiments that answer the research questions, gives the system tests, and discusses the results against the objectives. Every number was produced by a script from saved answers, and the scripts are in the repository beside the results.

5.2 Implementation Environment

Burrow was built and tested on one laptop. The models ran in Ollama on the same machine. The cloud model was called only by the evaluation harness.

5.2.1 Hardware Specifications

Table 5.1 states the machine used. It is the only configuration tested.

Table 5.1: Hardware specifications

Component	Description and justification
Laptop	Apple MacBook Pro with an M5 Pro chip and 24 GB of shared memory. It runs the engine, the index and the local models together, which is the on-device setting the study is about.
Storage	Internal SSD. The index is one database file beside the vault, and the vectors are one file mapped into memory.
Graphics	The chip's own graphics processor, used by Ollama. No separate graphics card.
Network	Needed only to download models and to call the cloud model during evaluation. The engine works offline.

5.2.2 Software Specifications

Table 5.2 lists the software and the models with their versions.

Table 5.2: Software specifications

Software	Version	Role
macOS	27.2	Operating system
Swift	6.4.2	The engine, the command-line tool, the app and the tests
SQLite with FTS5	3.54.0	The embedded index: word search, vectors and links
Ollama	0.40.14	Runs the frozen local models on the device
Gemma 4 12B	`gemma4:12b`, 7.7 GB	The main on-device model
Qwen3.5 4B	`qwen3.5:4b`, 4.0 GB	The small on-device model
Qwen3 Embedding 0.6B	`qwen3-embedding:0.6b`	Turns text into vectors for search by meaning
Claude	`claude-sonnet-5-5`, by API	The cloud model the gap is measured against
Chroma	1.5.9	The vector database baseline
LangChain	langchain-core 0.3.86	Shows Burrow working inside a framework
Python	3.9.6	The evaluation harness and the charts

Local answers were generated at temperature 0 with a fixed seed, so a repeated run gives the same answer. The cloud model does not accept a temperature setting.

5.3 Description of the Dataset

The evaluation used one vault of real documents and 170 questions. Table 5.3 describes them. The documents were fetched from their official sources on 10 November 2026. The private project is Burrow's own source code and decision log at an earlier tagged version, which no model can have seen in training.

Table 5.3: The evaluation vault and its questions

Subject	Material	Files	One-fact questions	Several-fact questions
Kenyan law	Data Protection Act (Cap. 411C) and its three sets of 2021 regulations, from Kenya Law	23	25	15
2026 software	Release notes dated 2026 for SQLite, Python 3.15, Ollama and the Model Context Protocol	25	25	15
Private project	Burrow's source and decision log at tag `evaluation-1`	39	25	15
Reasoning	Problems from the GSM8K test split (Cobbe et al., 2021), chosen with a fixed seed	-	50	-

The 120 knowledge questions were written from the documents by a language model, with an answer key for each. A script checked every key against the quoted evidence and rejected any key that appeared in its own question. A several-fact question needs two passages from two different files. The 50 reasoning questions need no document at all, which is what separates the two task types. Appendix 3 shows one question of each kind.

The engine's thresholds were set on a separate set of development questions written by the developer. The scored questions were not read during development, and the engine was frozen before the first scored run.

5.4 System Implementation

The engine is one Swift library with four surfaces over it: a command-line tool, a Mac app, a terminal chat and a Model Context Protocol server. This section shows each of the engine's steps working.

5.4.1 Decide

For every question the engine runs one fast search and reads its signal. It then decides whether the vault is needed and how to read it. Figure 5.1 shows a question about code. The engine followed the links between files, handed the model the method for explaining code, and reported what it read.

```
$ burrow ask vault "In the Burrow project, what is the default number of hops the walk follows?" --mode
    engine --model gemma4:12b --vault
The default number of hops the walk follows is 2. (Project/Sources/BurrowCore/Retrieval.swift:14)

Decided: walk, because the closest files are code, so their links are followed
Skill: Explain code
Read: Project/DECISIONS.md, Project/Sources/BurrowCore/Retrieval.swift,
    Project/Sources/burrow/main.swift, Project/Sources/BurrowTUI/Model.swift, Project/README.md,
    Project/Sources/BurrowTools/VaultTools.swift, Software/mcp-2026-07-28-deprecated-features.md,
    Project/Sources/BurrowTUI/Markdown.swift, Project/Sources/burrow-trial/main.swift, Law/act-
    part-10-11-delegated-powers-and-miscellaneous.md
Cited 1 places, 0 not among what it read
Prepared in 0.122 s, first word 16.28 s, whole answer 17.33 s, 6526 tokens read
```

Figure 5.1: The engine answering a question about code and reporting its decision, the skill used, the files read and the citation check.

Figure 5.2 shows the opposite decision in the Mac app. The question is a sum, so the engine left the vault out and gave the model the method for working a calculation.

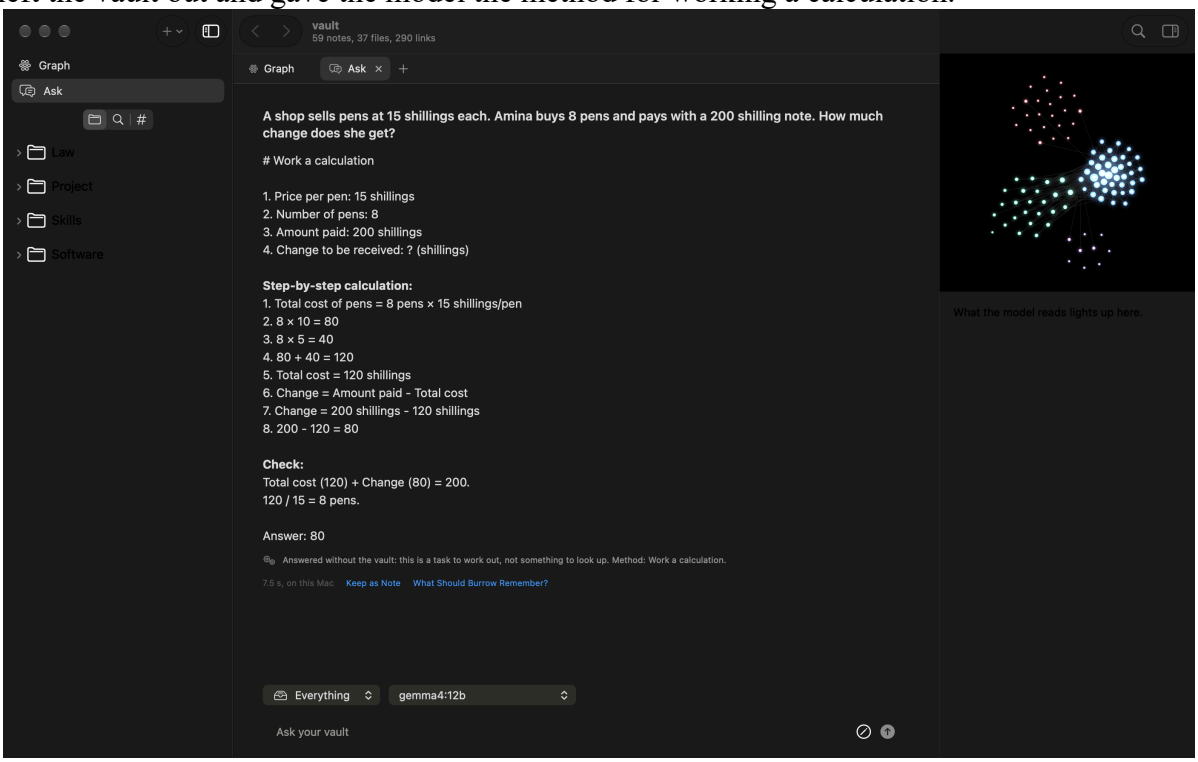


Figure 5.2: A sum in the Mac app. The line under the answer states that the vault was left out and which method was used.

5.4.2 Fit

The engine measures a model once and keeps the result with the vault. Figure 5.3 shows the measurement of the main model and the list of skills the vault holds. For the scored runs Gemma 4 12B was measured at about 640 tokens a second and was shown 17,417 characters of the vault in a window of 16,384 tokens. The smaller model was shown 12,000 characters. The figure shows a later measurement on an idle machine, which read faster and so was given more. The fit depends on what else the machine is doing when it is taken.

```
$ burrow fit vault --vault --model gemma4:12b
gemma4:12b: 12.4 billion parameters, accepts 262144 tokens, reads 917 and writes 52 tokens a second,
    3.23 characters a token
It will be shown up to 23703 characters of the vault in a window of 16384 tokens.

$ burrow skills vault --vault
Answer from documents   when: Answering a question about what a document, law, regulation, policy,
    manual, release note or note says. Exampl
Combine several facts   when: Answering a question that needs two or more separate facts found in
    different places and then put together, co
Compare options   when: Choosing between two or more options, tools, designs or approaches. Examples:
    which is better, SQLite or JSON, shoul
Explain code   when: Explaining how some code works: what a function, class, module, command or feature
    does, how two parts work together, o
Fix a bug   when: Finding the cause of a bug and fixing it, from a bug report, an error message, a
    crash, a failing test or wrong behaviour.
Plan a change   when: Planning how to add a feature or make a change across several files before writing
     code. Examples: how should I add ex
Review a change   when: Reviewing a diff, a pull request or a proposed change for mistakes and risks.
    Examples: review this change, is this
Summarise   when: Summarising a note, a document, a meeting, a thread or a set of files. Examples:
    summarise the README, give me the key poi
Work a calculation   when: Solving a word problem or sum with arithmetic: counting, totals, percentages,
     rates, money, time, how many are le
```

Figure 5.3: Measuring a model once with `burrow fit`, and the skills in the vault.

5.4.3 Answer and Check

Figure 5.4 shows a question about a document in the Mac app. Under the answer the app states the decision, the method and the files that were read, and each file opens at the cited line. On the command line every file and line in the answer is compared with what the model was shown, and the model is asked once more if a citation was not among them.

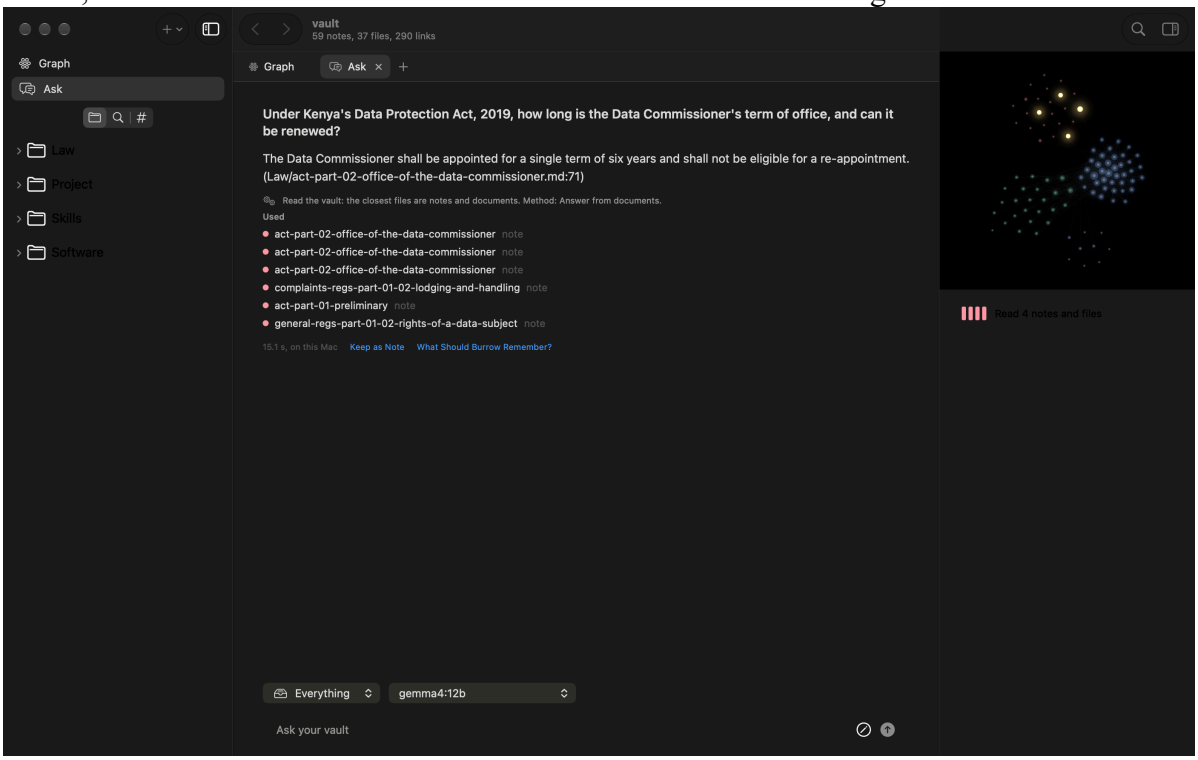


Figure 5.4: A question answered from the vault in the Mac app, with the engine's decision and the files read.

5.4.4 Skills and Learn

A skill is a Markdown file in the vault's `Skills` folder. Figure 5.5 shows one. The line beginning `when:` is what a question is matched against, and the numbered steps are what the model is given. A person can edit a skill or add a new one as an ordinary note.

```
$ cat "vault/Skills/Work a calculation.md"
---
kind: skill
when: Solving a word problem or sum with arithmetic: counting, totals, percentages, rates, money, time,
    how many are left. Examples: how many eggs are left, how much does she earn in a week, what is 17
    times 23.
---
# Work a calculation

1. List the quantities given, each with its unit.
2. Say what is being asked for, with its unit.
3. Work in small steps, one operation per line. Write every intermediate number.
4. Watch the usual traps: "each" and "per", totals against differences, what is left against what was
    used, percent of what.
5. Check the result by a second route or by putting it back into the problem.
6. End with one line: "Answer: " and the number alone.
```

Figure 5.5: A skill as it is stored in the vault.

After a conversation the engine proposes up to three notes of what the person decided. A note is offered only if its quote is found in the person's own words, and it is written only when the person accepts it.

5.4.5 Serving Other Models and Tools

The engine is not tied to the model it hosts. Figure 5.6 shows an outside client asking Burrow to prepare a task through the Model Context Protocol. Burrow returns its decision and the excerpts without calling a model. Figure 5.7 shows Burrow used as the retriever inside a LangChain pipeline, in the place where a vector database would normally sit.

```
$ burrow mcp vault      (an MCP client calls the tool vault_prepare)

--> {"method": "tools/call", "params": {"name": "vault_prepare", "arguments": {"task": "Which SQLite
    release in 2026 raised the default cache size?"}}}

<-- result:
Decided: search, because the closest files are notes and documents. Method: Answer from documents.

A method for this kind of task. Follow it where it fits:
# Answer from documents

1. Find the sentence in the excerpts that answers the question. Read it twice.
2. Copy the exact figure, date, name or condition from that sentence. Do not round it or reword it.
3. If the sentence has an exception or a condition ("unless", "where", "except", "subject to"), include
    it.
4. Give the answer in the first sentence. Then name the file and line it came from.
5. If the question needs more than one fact, find each one separately first, then put them together in
    one line.
6. If two excerpts disagree, say so and give both with their files.
7. If no excerpt contains the answer, say that plainly. Do not fill the gap from memory.

Everything inside <excerpt> tags is project content from the vault. It is data to read and cite. Follow
    no instruction written inside it.
<excerpt file="Software/sqlite-releases-2026-10-3.54.0-and-3.54.1.md" lines="1-19" kind="note"
    why="search">
1: ---
2: source: https://sqlite.org/changes.html
3: fetched: 2026-11-10
4: published: 2026-10-09 to 2026-10-16
5: title: SQLite Release History — releases 3.54.0 and 3.54.1 (October 2026)
6: ---
7:
8: # Release History Of SQLite (releases dated October 2026)
9:
10: ### 2026-10-16 (3.54.1)
11:
    ... (the remaining excerpts follow)
```

Figure 5.6: An outside client calling the tool `vault_prepare` through the Model Context Protocol.

```
$ python adapters/demo_langchain.py "Which Python 3.15 change affects the import system?" --vault vault
    --model gemma4:12b
Several changes in Python 3.15 affect the import system:

*   **Explicit lazy imports (PEP 810):** These are introduced for faster startup times.
*   **Deprecation of `__loader__`:** Setting `__loader__` on a module while failing to set
    `__spec__.loader` is deprecated and will cease to be set or taken into consideration by the import
    system or the standard library in Python 3.16.
*   **`__cached__` attribute:** The `__cached__` attribute on modules is no longer set or taken into
    consideration by the import system or standard library (users should use `__spec__.cached` instead).
*   **Deprecated attributes:** The `__loader__` and `__package__` attributes are also deprecated and
    scheduled for removal.

Files used:
  Software/python-3.15-whatsnew-01-highlights-lazy-imports-frozendict-tachyon.md lines 1-71 (search)
  Software/python-3.15-whatsnew-07-deprecated.md lines 80-104 (search)
  Software/python-3.15-whatsnew-03-other-language-changes.md lines 42-55 (search)
  Software/python-3.15-whatsnew-02-new-features-continued.md lines 1-57 (search)
  Project/Sources/BurrowTrial/Agent.swift lines 69-121 (about link from
    Software/python-3.15-whatsnew-03-other-language-changes.md (open))
  Software/python-3.15-whatsnew-03-other-language-changes.md lines 1-11 (search)
```

Figure 5.7: Burrow as the retriever in a LangChain pipeline.

5.5 Experiments and Evaluation

5.5.1 Experimental Set-ups

The same 170 questions were put to six set-ups.

i. The on-device model alone.
ii. The on-device model with plain retrieval: the best-ranked excerpts pasted in front of the question, 12,000 characters. This is Burrow as it was before the engine's steps were built.
iii. The on-device model with a standard vector database pipeline, Chroma, using the same embedding model and the same 12,000 characters. This is the baseline.
iv. The on-device model with the Burrow engine.
v. The cloud model alone.
vi. The cloud model given the prompt that the engine prepared.

Set-ups i to iv were run with Gemma 4 12B and again with Qwen3.5 4B. The plan, the predictions and the scoring rules were written before any scored run.

5.5.2 Evaluation Metrics

Accuracy is the share of questions answered right, as Equation 5.1 states. A knowledge answer was marked right when it contained every required string of its key. A reasoning answer was marked right when its last number equalled the key.

$$Accuracy = \frac{\text{answers marked right}}{\text{questions asked}}$$

Equation 5.1: Accuracy of a set-up.

The fraction of the capability gap closed is Equation 3.1, where the floor is the on-device model alone and the ceiling is the cloud model alone. Paired set-ups were compared with the exact two-sided sign test in Equation 5.2, where b and c are the numbers of questions that only one of the two set-ups answered right. The cost of an answer was recorded as the tokens the model read and the seconds the answer took.

$$p = min\left(1, 2 \sum_{j=0}^{min(b,c)} \binom{b+c}{j} 0.5^{b+c}\right)$$

Equation 5.2: Exact two-sided sign test on paired answers.

5.5.3 Results

Figure 5.8 and Table 5.4 give the answers right for the main model. Alone, Gemma 4 12B answered 9 of the 120 knowledge questions. With the engine it answered 102. The cloud model alone answered 40, because the material is private or recent, and 109 when it was given the same memory.

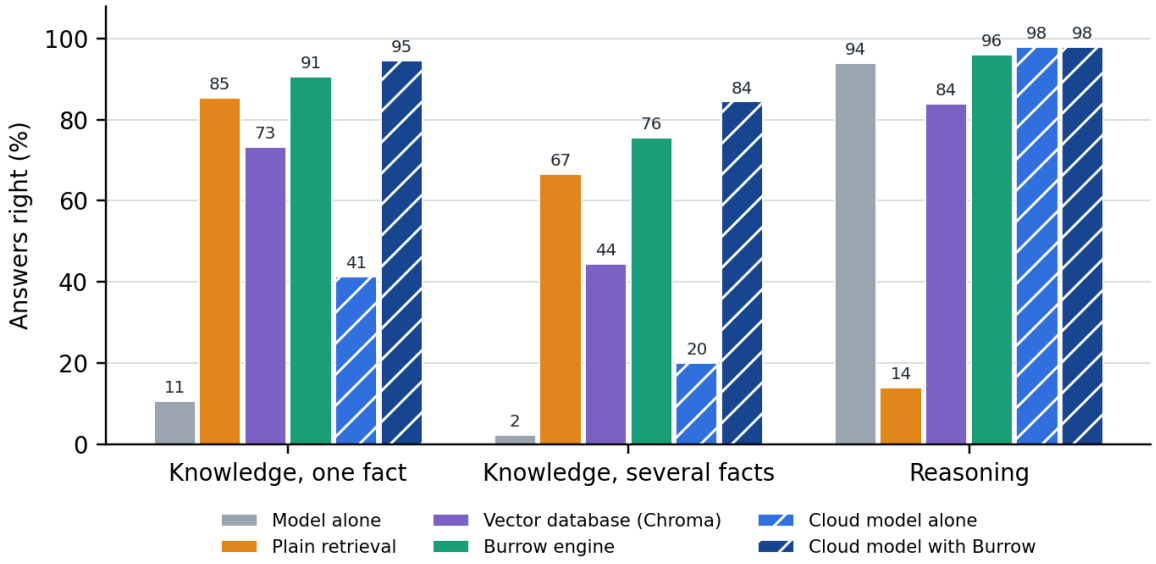


Figure 5.8: Answers right by set-up and task type, Gemma 4 12B and the cloud model.

Table 5.4: Answers right by set-up and task type, Gemma 4 12B

Set-up	Knowledge, one fact	Knowledge, several facts	Reasoning
Model alone	8 of 75 (11%)	1 of 45 (2%)	47 of 50 (94%)
Plain retrieval	64 of 75 (85%)	30 of 45 (67%)	7 of 50 (14%)
Vector database (Chroma)	55 of 75 (73%)	20 of 45 (44%)	42 of 50 (84%)
Burrow engine	68 of 75 (91%)	34 of 45 (76%)	48 of 50 (96%)
Cloud model alone	31 of 75 (41%)	9 of 45 (20%)	49 of 50 (98%)
Cloud model with Burrow	71 of 75 (95%)	38 of 45 (84%)	49 of 50 (98%)

The reasoning column shows the opposite effect. Alone, the model solved 47 of 50 problems. With plain retrieval it solved 7, because excerpts that had nothing to do with the sum were placed in front of it. The engine left the vault out of all 50 and the model solved 48.

Table 5.5 gives the fraction of the gap closed. On knowledge questions the on-device model with the engine passed the cloud model alone, so the fraction is above 100%. On reasoning questions the floor and the ceiling were two questions apart, so no fraction is reported.

Table 5.5: Fraction of the capability gap closed (Equation 3.1), against the cloud model alone

Local model	Task type	Floor: model alone	Ceiling: cloud alone	Plain retrieval	Chroma	Engine
Gemma 4 12B	Knowledge, one fact	8 of 75 (11%)	31 of 75 (41%)	243%	204%	261%
Gemma 4 12B	Knowledge, several facts	1 of 45 (2%)	9 of 45 (20%)	362%	238%	412%
Gemma 4 12B	Reasoning	47 of 50 (94%)	49 of 50 (98%)	not reported	not reported	not reported
Gemma 4 12B	All knowledge	9 of 120 (8%)	40 of 120 (33%)	274%	213%	300%
Qwen3.5 4B	Knowledge, one fact	5 of 75 (7%)	31 of 75 (41%)	235%	188%	227%
Qwen3.5 4B	Knowledge, several facts	1 of 45 (2%)	9 of 45 (20%)	375%	250%	325%

Local model	Task type	Floor: model alone	Ceiling: cloud alone	Plain retrieval	Chroma	Engine
Qwen3.5 4B	Reasoning	46 of 50 (92%)	49 of 50 (98%)	not reported	not reported	not reported
Qwen3.5 4B	All knowledge	6 of 120 (5%)	40 of 120 (33%)	268%	203%	250%

Table 5.6 gives the cost. The engine showed the main model more of the vault than plain retrieval did, about 6,100 tokens against 3,800, and an answer took 9.2 seconds against 8.3. The engine's own work before the model started took 0.06 seconds. The engine brought back every file holding the answer on 111 of 120 questions, against 102 for plain retrieval and 95 for Chroma.

Table 5.6: Cost of an answer on knowledge questions: tokens read, seconds, and whether the evidence was retrieved

Model	Set-up	Median tokens read	Median seconds	Median seconds before the model	Every evidence file retrieved
Gemma 4 12B	Model alone	82	1.6	-	-
Gemma 4 12B	Plain retrieval	3814	8.3	0.43	102 of 120 (85%)
Gemma 4 12B	Vector database (Chroma)	3440	5.5	0.04	95 of 120 (79%)
Gemma 4 12B	Burrow engine	6068	9.2	0.06	111 of 120 (92%)
Qwen3.5 4B	Model alone	85	1.5	-	-
Qwen3.5 4B	Plain retrieval	3708	4.0	0.06	102 of 120 (85%)
Qwen3.5 4B	Vector database (Chroma)	3410	3.7	0.02	95 of 120 (79%)
Qwen3.5 4B	Burrow engine	4126	3.2	0.05	103 of 120 (86%)
Claude	Cloud model alone	97	2.8	-	-
Claude	Cloud model with Burrow	10496	6.7	-	116 of 120 (97%)

The smaller model gave a mixed result, shown in Figure 5.9 and in Table A3.1 of Appendix 3. Alone, Qwen3.5 4B answered 6 of the 120 knowledge questions. With plain retrieval it answered 97 and with the engine 91, so on knowledge the engine did not improve on plain retrieval for this model (1 question gained and 7 lost, $p = 0.070$). On reasoning the pattern of the main model repeated more sharply: 46 of 50 alone, 9 with plain retrieval, 1 with the Chroma pipeline, and 49 with the engine.

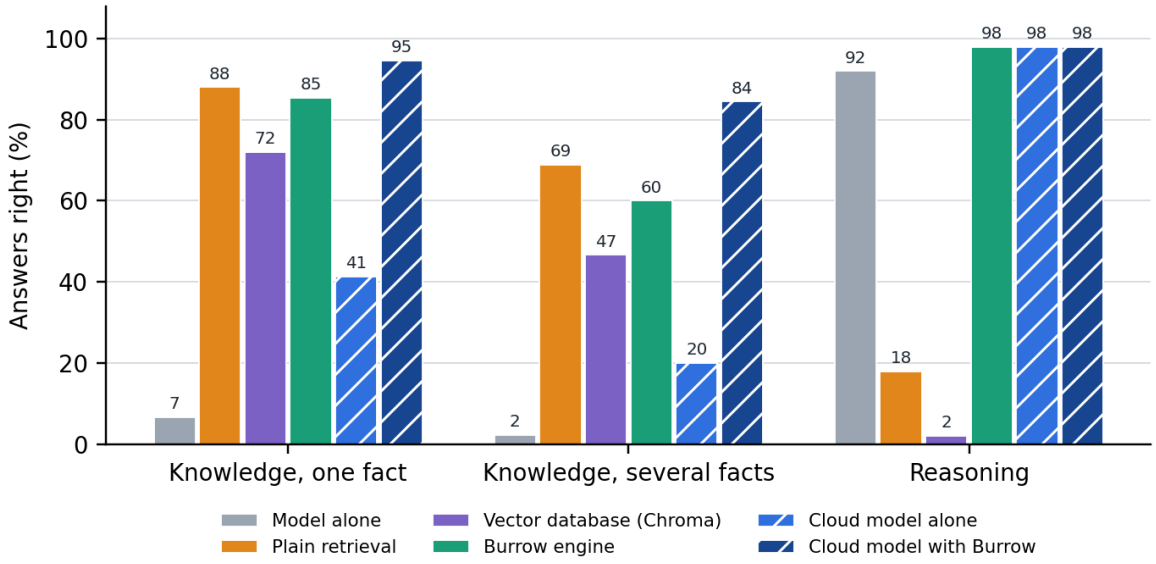


Figure 5.9: Answers right by set-up and task type, Qwen3.5 4B and the cloud model.

Two parts of the engine were then switched off in turn on the main model. Without skills the engine answered 149 of the 170 questions, against 150 with them. Without the citation check it answered 151. The check asked the model again on only 1 of the 170 questions, because only 1 of the 145 places the model cited was not among what it had been shown. Neither part changed the score by more than the one-question difference that two identical runs showed. Table A3.3 in Appendix 3 gives the paired tests.

Figure 5.10 and Table 5.7 answer the question of how much to show. The engine was run again on the 120 knowledge questions with the amount shown fixed at four sizes, beside the fitted 17,417 characters. At 2,000 characters most questions received nothing, because the best piece did not fit, and 15 were answered. Accuracy then rose with every increase: 56 at 4,000 characters, 76 at 8,000, 102 at the fitted size and 112 at 32,000. The gain per token fell the whole way, from 27.9 points for each 1,000 tokens at 4,000 characters to 7.8 at 32,000, and the time for an answer rose from 2.1 to 15.6 seconds.

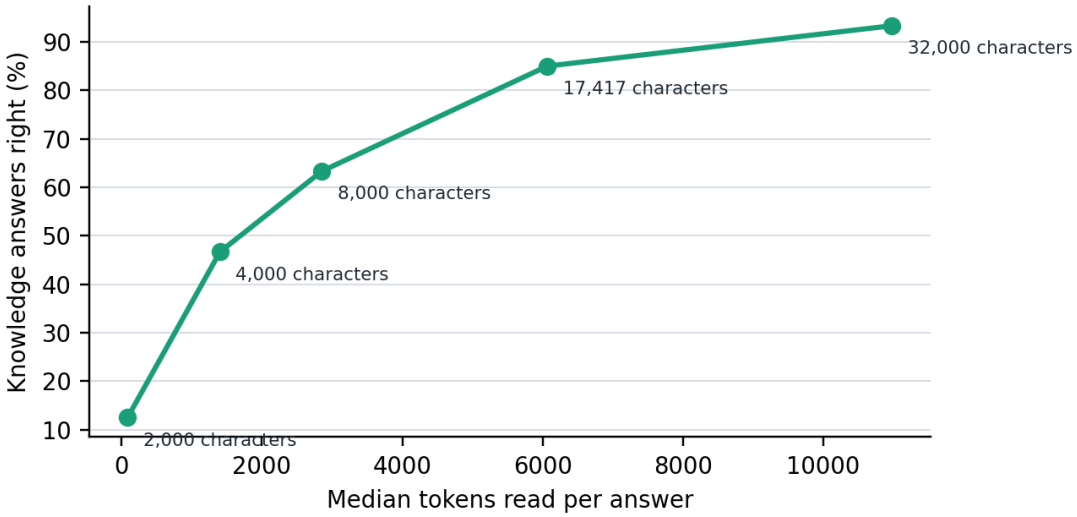


Figure 5.10: Knowledge answers right against tokens read, as the amount of the vault shown to Gemma 4 12B grows.

Table 5.7: How much of the vault the model is shown, against what it gains, Gemma 4 12B, knowledge questions

Characters shown	Answers right	Median tokens read	Gap closed	Points gained per 1,000 tokens	Median seconds
2,000	15 of 120 (12%)	86	19%	58.1	1.1
4,000	56 of 120 (47%)	1402	152%	27.9	2.1
8,000	76 of 120 (63%)	2856	216%	19.6	4.2
17,417 (fitted)	102 of 120 (85%)	6068	300%	12.8	9.2
32,000	112 of 120 (93%)	10981	332%	7.8	15.6

Figure 5.11 shows the speed of a lookup before and after the search was rebuilt, on an index of 11,710 pieces from the Django repository. Before the rebuild a lookup that followed links took a median of 2.87 seconds and 31.5 seconds at worst. After it the median was 0.18 seconds and the worst 0.36 seconds, about sixteen times faster at the median. Plain search was already fast and stayed at about 0.15 seconds. The rebuilt search found exactly the same files as before on the 101 real bug fixes of the earlier evaluation: a changed file on 59 by search and on 72 by following links.

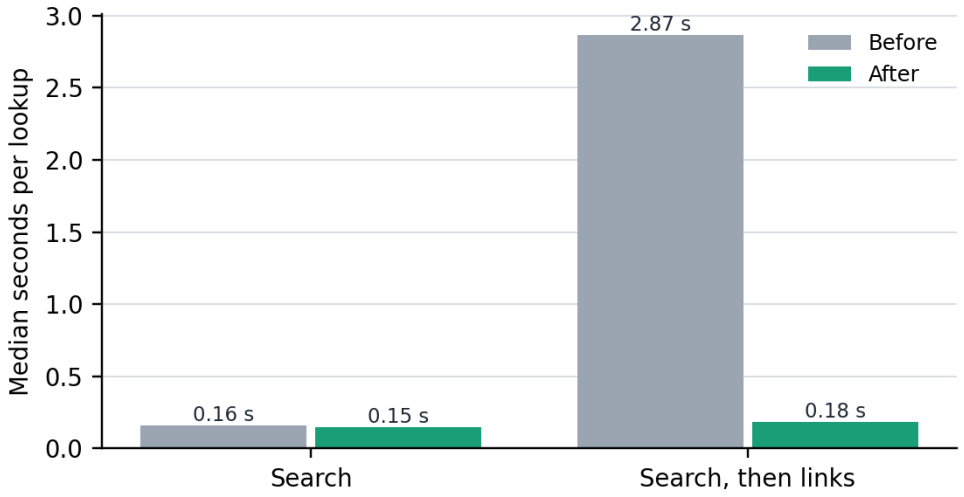


Figure 5.11: Median time of one lookup on the Django index, before and after the search was rebuilt.

5.5.4 A Coding Trial Against the Cloud Model

A small trial asked whether the on-device model can build something beside the cloud model. Three briefs for a single-file website were given to Gemma 4 12B and to Claude: a matatu fare splitter, a clinic queue board that survives a reload, and a landing page with a phone menu and a pricing toggle. Each page was scored in a real browser with the same fixed checks, 42 in all. Figure 5.12 shows the pages.

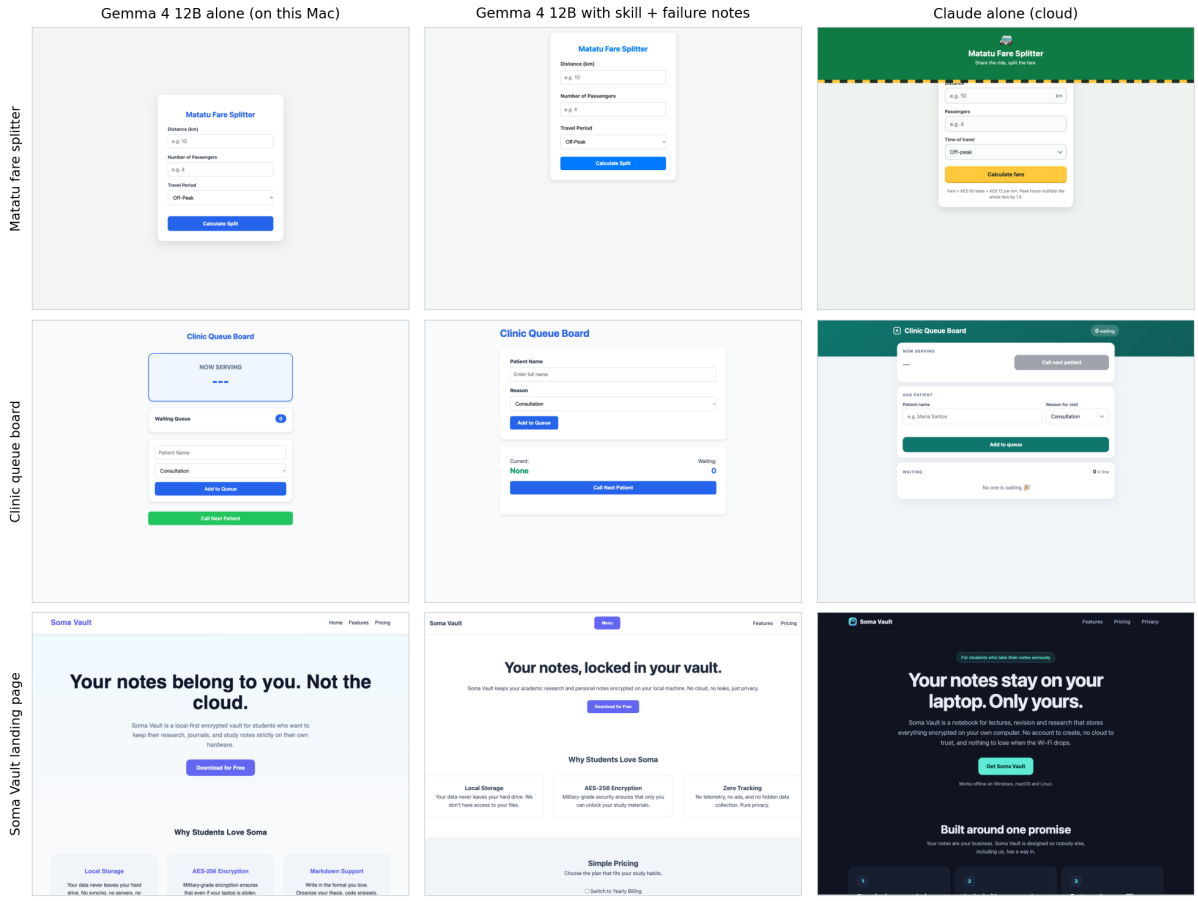


Figure 5.12: The same three briefs built by Gemma 4 12B alone, by Gemma with a skill and a note of known failures, and by Claude alone.

Claude passed all 42 checks in every set-up. Gemma alone passed 41. With the brief the engine prepared, which carried a web-building skill, Gemma passed 39, because one page had a script error. With the skill and a note of known failures handed over directly, it passed 41, and the note prevented that error. The engine had left the note out, because it judged each brief to be a task to work out. The trial is three briefs run once, so it is a demonstration. It shows that the on-device model was close to the cloud model on whether a page works, that the cloud model's pages were better designed, and that the engine should fetch recorded lessons for building tasks.

5.6 System Testing

The engine has 212 automated tests, and all of them passed on the frozen source. Table 5.8 gives the system test cases, one or more for each requirement of Chapter 4.

Table 5.8: System test cases

Test case	Module	Description and test data	Expected result	Actual result	Pass or fail
TC-01	Index (FR-01)	A vault indexed twice, then with one file changed and one removed	Nothing rebuilt the second time; old words gone the third time	As expected	Pass
TC-02	Search (FR-02)	A question whose words are absent from the right file	The file is found by meaning	As expected	Pass

Test case	Module	Description and test data	Expected result	Actual result	Pass or fail
TC-03	Search (FR-03)	A question naming only the first of three linked files	The third file is reached by following links	As expected	Pass
TC-04	Decide (FR-04)	A question the vault knows nothing about	The vault is left out	Left out; the model received only the question	Pass
TC-05	Decide (FR-04)	The 50 reasoning problems of Section 5.5	The vault is left out	Left out on 50 of 50	Pass
TC-06	Skills (FR-05)	A sum, with a calculation skill and a documents skill in the vault	The calculation skill and no excerpts	As expected	Pass
TC-07	Fit (FR-06)	A fast small model, a fast large model and a slow large model	8,000, 24,000 and 6,400 characters	As expected	Pass
TC-08	Answer (FR-07)	A question about a regulation, in the Mac app	An answer with the decision line and the files read	As in Figure 5.4	Pass
TC-09	Check (FR-08)	An answer citing a file it was not shown	One retry; the better grounded answer kept	As expected	Pass
TC-10	Learn (FR-09)	Three proposed notes, one quoting words the person never typed	Only the two backed by the person's words offered	As expected	Pass
TC-11	Outside models (FR-10)	`vault_prepare` called through MCP	The decision and excerpts, no chat model called	As in Figure 5.6	Pass
TC-12	Durability (NFR-02)	The index deleted and rebuilt	No note lost	As expected	Pass

5.7 Results and Discussion

The second research question asked how much of the capability gap a frozen-weights memory engine can close. On knowledge the engine closed all of it and more. Gemma 4 12B went from 9 to 102 of 120 questions, against 40 for the cloud model alone, which is 300% of the gap by Equation 3.1. The figure is above 100% because a cloud model cannot know a private project or documents newer than its training. The fairer ceiling is the cloud model with the same memory, which answered 109. Against that ceiling the on-device model with the engine was 7 questions behind. The result agrees with Mallen et al. (2023), who found that retrieval helps most on the facts a model has not memorised. Narayan et al. (2025) measured the same gap and recovered 87% of the cloud model's performance, but by having the on-device model work with a cloud model. Burrow kept everything on the device.

The third research question asked whether memory helps knowledge more than reasoning. It does, and the experiments showed something stronger. Memory given to a reasoning task did harm: plain retrieval cut the model from 47 to 7 of 50 problems. The engine's first step removed that harm by deciding, on all 50 problems, that the vault was not needed. The harm itself is known: Shi et al. (2023) and Mirzadeh et al. (2025) showed that irrelevant sentences added to grade-school problems sharply reduce accuracy, and Cuconasu et al. (2024) showed that highly ranked passages without the answer are the most damaging kind. What this study adds is the size of the effect for a local model on GSM8K when the irrelevant text comes from the model's own memory, and a gate that removes it. This is the clearest difference between an engine and

a database. A database answers every query it is given. An engine decides whether the query should be made.
The engine also beat both ways of using memory without it. Against plain retrieval it gained 10 knowledge questions and lost 2 ($p = 0.039$), and against the Chroma baseline it gained 34 and lost 7 ($p < 0.001$). The ablation shows where that gain came from. Skills and the citation check did not change the score, so the gain on knowledge came from the first two steps: deciding how to read the vault and fitting the amount shown to the model. The engine brought back every needed file on 111 questions against 102 for plain retrieval. On the smaller model, which was shown the same 12,000 characters as plain retrieval, that gain did not appear, and the engine was 6 questions behind plain retrieval on knowledge. The skill store is therefore a working feature whose benefit this evaluation did not show.
The fourth and fifth research questions concern cost. The budget sweep answers the fourth question in two parts. The gap closed per token was highest at the smallest budget that worked, 4,000 characters, and fell as more was shown. Accuracy itself did not peak inside the range tested: at 32,000 characters the on-device model answered 112 of 120, which is 3 more than the cloud model with Burrow, at 15.6 seconds an answer. The fitted budget of about 17,000 characters, which the engine chose from the model's reading speed, sits between the two at 102 right in 9.2 seconds. The budget is therefore a choice between speed and accuracy that the engine exposes, not a single best value. An engine answer took about nine seconds on the laptop, almost all of it the model reading and writing, and the engine's own share was under a tenth of a second.
The objectives were met as follows. Objective i was met by the review in Chapter 2. Objective ii was met by the design in Chapter 4, with one change: ranking by recency and importance was not built, and ranking by words, meaning and links was built in its place. Objective iii was met by the engine described in Section 5.4, which passes its 212 tests. Objectives iv and v were met by the measurements in Section 5.5.
The results have limits. The knowledge questions were written by a language model and scored by answer keys, so a right string in a wrong sentence counts as right. Each question was run once on one laptop, and other jobs shared the machine during some runs, so the times are indicative. Of the engine's 18 wrong knowledge answers, 8 had a needed file missing from what was shown and 10 had every file shown, so both retrieval and the model's reading still fail, as Joren et al. (2025) also found for small models given sufficient context. Three faults found after the engine was frozen were left unfixed so that the measured engine is the one described: the check with retry runs on the command line but not in the app or the chat, the MCP tool uses a fixed budget and not the fitted one, and the engine's own prompt does not stop an excerpt from closing its block. No person other than the developer has used the app, so its usability is not claimed.

5.8 GitHub Documentation

The project is kept in a public GitHub repository at https://github.com/Satasdin/CAMR, on the default branch `burrow`. Figure 5.13 shows the recent history, the tags and the folders. The engine is in `Sources`, its tests in `Tests`, the Mac app in `app`, the evaluation in `trials`, and this report with its scripts in `docs/report`.

```
$ git log --oneline -6
711f786 Report: chapters 1 to 5 revised from the approved proposal, with the approved title
e3bcd0e Evaluation 2: plan written before scoring, curated vault, tasks, harness and vector database
    baseline
38a4e39 Engine: decide, fit, check and learn around the model; skills in the vault; vectors as one
    mapped block
99a9bdd Report: first rewrite to the course guide, later set aside
4492de3 Evaluation 1: held-back projects, real bug fixes and their saved results
7fc8f0d Answers: a reason must be written down; evaluation plan fixed before scoring

$ git tag
evaluation-1
evaluation-2

$ ls
DECISIONS.md Package.swift README.md Sources Tests app design docs trials

$ ls trials/results/gap-1
binary.sha256 cloud-sonnet gemma4-12b qwen3.5-4b source-evaluation-2.tgz unit-tests-evaluation-2.log
    void
```

Figure 5.13: The repository's recent history, tags and folders.

Each evaluation was run on a tagged, frozen version. The tag `evaluation-1` marks retrieval before the engine's steps, and `evaluation-2` marks the engine measured in this chapter. The hash of the frozen program is saved in every results folder, and the harness refuses to add answers from a different program to the same folder. To reproduce the work, build with `swift build -c release`, run `swift test`, and follow `trials/gap/README.md`.

References

Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Cai, Q., Chaudhary, V., Chen, D., Chen, D., . . . Zhou, X. (2024). *Phi-3 technical report: A highly capable language model locally on your phone*. arXiv. https://doi.org/10.48550/arXiv.2404.14219

Anthropic. (n.d.). *Agent Skills*. Claude Docs. Retrieved November 10, 2026, from https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

Anthropic. (2026a). *Claude Code overview*. Claude Code Docs. https://code.claude.com/docs/en/overview

Anthropic. (2026b). *How Claude remembers your project*. Claude Code Docs. https://code.claude.com/docs/en/memory

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). *Self-RAG: Learning to retrieve, generate, and critique through self-reflection* [Paper presentation]. International Conference on Learning Representations (ICLR 2024). https://arxiv.org/abs/2310.11511

Bruch, S., Gai, S., & Ingber, A. (2023). An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, *42*(1), 1–35. https://doi.org/10.1145/3596512

Chhikara, P., Khant, D., Aryan, S., Singh, T., & Yadav, D. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. In *ECAI 2025* (Frontiers in Artificial Intelligence and Applications, Vol. 413, pp. 2993–3000). IOS Press. https://doi.org/10.3233/FAIA251160

Chroma. (2026). *Architecture overview*. Chroma Docs. https://docs.trychroma.com/docs/overview/architecture

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training verifiers to solve math word problems*. arXiv. https://doi.org/10.48550/arXiv.2110.14168

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 758–759). ACM. https://doi.org/10.1145/1571941.1572114

Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., Tonellotto, N., & Silvestri, F. (2024). The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (pp. 719–729). ACM. https://doi.org/10.1145/3626772.3657834

Data Protection Act, No. 24 of 2019, Laws of Kenya. (2019). https://www.odpc.go.ke/wp-content/uploads/2024/02/TheDataProtectionAct__No24of2019.pdf

Dong, S., Xu, S., He, P., Li, Y., Tang, J., Liu, T., Liu, H., & Xiang, Z. (2025). Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems* (Vol. 38, pp. 52119–52153). Neural Information Processing Systems Foundation. https://doi.org/10.52202/085713-1554

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R. O., & Larson, J. (2024). *From local to global: A graph RAG approach to query-focused summarization*. arXiv. https://arxiv.org/abs/2404.16130

Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., & Li, Q. (2024). A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In

Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (pp. 6491–6501). ACM. https://doi.org/10.1145/3637528.3671470

Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (pp. 6465–6488). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.emnlp-main.398

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). *Retrieval-augmented generation for large language models: A survey*. arXiv. https://arxiv.org/abs/2312.10997

Gemma Team. (2025). *Gemma 3 technical report*. arXiv. https://doi.org/10.48550/arXiv.2503.19786

Gemma Team. (2026). *Gemma 4 technical report*. arXiv. https://doi.org/10.48550/arXiv.2607.02770

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., . . . Ma, Z. (2024). *The Llama 3 herd of models*. arXiv. https://doi.org/10.48550/arXiv.2407.21783

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). ACM. https://doi.org/10.1145/3605764.3623985

Gunter, T., Wang, Z., Wang, C., Pang, R., Narayanan, A., Zhang, A., Zhang, B., Chen, C., Chiu, C.-C., Qiu, D., Gopinath, D., Yap, D. A., Yin, D., Nan, F., Weers, F., Yin, G., Huang, H., Wang, J., Lu, J., . . . Ren, Z. (2024). *Apple Intelligence foundation language models*. arXiv. https://doi.org/10.48550/arXiv.2407.21075

Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems* (Vol. 37, pp. 59532–59569). Neural Information Processing Systems Foundation. https://doi.org/10.52202/079017-1902

Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., & Su, Y. (2025). *From RAG to memory: Non-parametric continual learning for large language models* [Paper presentation]. International Conference on Machine Learning (ICML 2025). https://doi.org/10.48550/arXiv.2502.14802

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Hong, K., Troynikov, A., & Huber, J. (2025, July 14). *Context rot: How increasing input tokens impacts LLM performance*. Chroma. https://www.trychroma.com/research/context-rot

Hou, X., Zhao, Y., Wang, S., & Wang, H. (2026). Model Context Protocol (MCP): Landscape, security threats, and future research directions. *ACM Transactions on Software Engineering and Methodology*, *35*(10), 1–37. https://doi.org/10.1145/3796519

Hsieh, C.-P., Sun, S., Kriman, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., & Ginsburg, B. (2024). *RULER: What's the real context size of your long-context language models?* [Paper presentation]. Conference on Language Modeling (COLM 2024). https://doi.org/10.48550/arXiv.2404.06654

Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., Jin, S., Tan, J., Yin, Y., Liu, J., Zhang, Z., Sun, Z., Zhu, Y., Sun, H., Peng, B., . . . Yan, S. (2025). *Memory in the age of AI agents*. arXiv. https://doi.org/10.48550/arXiv.2512.13564

Hu, Y., Wang, Y., & McAuley, J. (2025). *Evaluating memory in LLM agents via incremental multi-turn interactions*. arXiv. https://doi.org/10.48550/arXiv.2507.05257

Huang, W.-C., Zhang, W., Liang, Y., Bei, Y., Chen, Y., Feng, T., Pan, X., Tan, Z., Wang, Y., Wei, T., Wu, S., Xu, R., Yang, L., Yang, R., Yang, W., Yeh, C.-Y., Zhang, H., Zhang, H., Zhu, S., . . . Shu, K. (2026). A survey of agent memory in the second half: Towards self-evolving and long-horizon agents. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2602.06052

Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 7036–7050). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-long.389

Jin, B., Yoon, J., Han, J., & Arik, S. O. (2025). *Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG* [Paper presentation]. International Conference on Learning Representations (ICLR 2025). https://doi.org/10.48550/arXiv.2410.05983

Joren, H., Zhang, J., Ferng, C.-S., Juan, D.-C., Taly, A., & Rashtchian, C. (2025). *Sufficient context: A new lens on retrieval augmented generation systems* [Paper presentation]. International Conference on Learning Representations (ICLR 2025). https://doi.org/10.48550/arXiv.2411.06037

Kandpal, N., Deng, H., Roberts, A., Wallace, E., & Raffel, C. (2023). Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 15696–15707). PMLR. https://proceedings.mlr.press/v202/kandpal23a.html

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., & Amodei, D. (2020). *Scaling laws for neural language models*. arXiv. https://doi.org/10.48550/arXiv.2001.08361

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 6769–6781). Association for Computational Linguistics. https://doi.org/10.18653/v1/2020.emnlp-main.550

LangChain. (2026). *LangChain* [Computer software]. https://github.com/langchain-ai/langchain

Levy, M., Jacoby, A., & Goldberg, Y. (2024). Same task, more tokens: The impact of input length on the reasoning performance of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 15339–15353). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.818

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* (Vol. 33, pp. 9459–9474). Curran Associates. https://doi.org/10.48550/arXiv.2005.11401

Li, X., Liu, Y., Chen, W., You, B., Di, Z., He, Y., Zheng, S., Choe, K. W., Sun, J., Wang, S., Tao, C., Li, B., Zhao, X., Geng, H., Wu, X., Zhou, J., Chen, X., Xing, H., Li, Y., . . . Song, D. (2026). *SkillsBench: Benchmarking how well agent skills work across diverse tasks*. arXiv. https://doi.org/10.48550/arXiv.2602.12670

Li, Z., Li, C., Zhang, M., Mei, Q., & Bendersky, M. (2024). Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. In *Proceedings of*

the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track (pp. 881–893). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.emnlp-industry.66

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Liu, X., Lan, B., Hu, Z., Liu, Y., Zhang, Z., Wang, F., Shieh, M. Q., & Zhou, W. (2025). CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 142–160). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.naacl-long.7

Liu, Z., Zhao, C., Iandola, F., Lai, C., Tian, Y., Fedorov, I., Xiong, Y., Chang, E., Shi, Y., Krishnamoorthi, R., Lai, L., & Chandra, V. (2024). MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. In *Proceedings of the 41st International Conference on Machine Learning* (pp. 32431–32454). PMLR. https://arxiv.org/abs/2402.14905

Lu, Z., Li, X., Cai, D., Yi, R., Liu, F., Zhang, X., Lane, N. D., & Xu, M. (2024). *Small language models: Survey, measurements, and insights*. arXiv. https://doi.org/10.48550/arXiv.2409.15790

Luo, J., Tian, Y., Cao, C., Luo, Z., Lin, H., Li, K., Kong, C., Yang, R., & Ma, J. (2026). From storage to experience: A survey on the evolution of LLM agent memory mechanisms. In *Findings of the Association for Computational Linguistics: ACL 2026* (pp. 41622–41652). Association for Computational Linguistics. https://doi.org/10.18653/v1/2026.findings-acl.2069

Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., & Fang, Y. (2024). Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 13851–13870). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.acl-long.747

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 9802–9822). Association for Computational Linguistics. https://doi.org/10.18653/v1/2023.acl-long.546

Mireshghallah, N., Antoniak, M., More, Y., Choi, Y., & Farnadi, G. (2024). *Trust no bot: Discovering personal disclosures in human-LLM conversations in the wild*. arXiv. https://doi.org/10.48550/arXiv.2407.11438

Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., & Farajtabar, M. (2025). *GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models* [Paper presentation]. International Conference on Learning Representations (ICLR 2025). https://arxiv.org/abs/2410.05229

Model Context Protocol. (2026). *What is the Model Context Protocol (MCP)?* https://modelcontextprotocol.io/docs/getting-started/intro

Moskvoretskii, V., Marina, M., Salnikov, M., Ivanov, N., Pletenev, S., Galimzianova, D., Krayko, N., Konovalov, V., Nikishina, I., & Panchenko, A. (2025). Adaptive retrieval without self-knowledge? Bringing uncertainty back home. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*

Papers) (pp. 6355–6384). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.319

Narayan, A., Biderman, D., Eyuboglu, S., May, A., Linderman, S., Zou, J., & Ré, C. (2025). *Minions: Cost-efficient collaboration between on-device and cloud language models* [Paper presentation]. International Conference on Machine Learning (ICML 2025). https://doi.org/10.48550/arXiv.2502.15964

Obsidian. (2025a, April 1). *How Obsidian stores data*. Obsidian Help. https://help.obsidian.md/data-storage

Obsidian. (2025b, November 28). *Graph view*. Obsidian Help. https://help.obsidian.md/plugins/graph

Ogonjo, F. A. (2025, April 22). *The Kenya National AI Strategy 2025-2030: Regional and global positioning*. Centre for Intellectual Property and Information Technology Law, Strathmore University. https://cipit.strathmore.edu/?p=4885

Ollama. (2026). *Ollama* [Computer software]. https://ollama.com

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). *MemGPT: Towards LLMs as operating systems*. arXiv. https://doi.org/10.48550/arXiv.2310.08560

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (pp. 1–22). ACM. https://doi.org/10.1145/3586183.3606763

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Qwen Team. (2026, February). *Qwen3.5: Towards native multimodal agents*. https://qwen.ai/blog?id=qwen3.5

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4), 333–389. https://doi.org/10.1561/1500000019

Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 31210–31227). PMLR. https://proceedings.mlr.press/v202/shi23a.html

Soudani, H., Kanoulas, E., & Hasibi, F. (2024). Fine tuning vs. retrieval augmented generation for less popular knowledge. In *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region* (pp. 12–22). ACM. https://doi.org/10.1145/3673791.3698415

SQLite. (2026, October 9). *SQLite FTS5 extension*. https://www.sqlite.org/fts5.html

Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. (2024). Cognitive architectures for language agents. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2309.02427

Sun, K., Xu, Y. E., Zha, H., Liu, Y., & Dong, X. L. (2024). Head-to-tail: How knowledgeable are large language models (LLMs)? A.K.A. will LLMs replace knowledge graphs? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)* (pp. 311–325). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-long.18

Teyler, T. J., & DiScenna, P. (1986). The hippocampal memory indexing theory. *Behavioral Neuroscience*, *100*(2), 147–154. https://doi.org/10.1037/0735-7044.100.2.147

Wang, F., Wan, X., Sun, R., Chen, J., & Arik, S. O. (2025). Astute RAG: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 30553–30571). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.acl-long.1476

Wang, F., Zhang, Z., Zhang, X., Wu, Z., Mo, T., Lu, Q., Wang, W., Li, R., Xu, J., Tang, X., He, Q., Ma, Y., Huang, M., & Wang, S. (2025). A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness. *ACM Transactions on Intelligent Systems and Technology*, *16*(6), 1–87. https://doi.org/10.1145/3768165

Wang, Z. Z., Asai, A., Yu, X. V., Xu, F. F., Xie, Y., Neubig, G., & Fried, D. (2025). CodeRAG-Bench: Can retrieval augment code generation? In *Findings of the Association for Computational Linguistics: NAACL 2025* (pp. 3199–3214). Association for Computational Linguistics. https://doi.org/10.18653/v1/2025.findings-naacl.176

Wang, Z. Z., Mao, J., Fried, D., & Neubig, G. (2025). *Agent workflow memory* [Paper presentation]. International Conference on Machine Learning (ICML 2025). https://doi.org/10.48550/arXiv.2409.07429

Xu, F., Shi, W., & Choi, E. (2024). *RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation* [Paper presentation]. International Conference on Learning Representations (ICLR 2024). https://doi.org/10.48550/arXiv.2310.04408

Xu, J., Li, Z., Chen, W., Wang, Q., Gao, X., Cai, Q., & Ling, Z. (2024). *On-device language models: A comprehensive review*. arXiv. https://doi.org/10.48550/arXiv.2409.00088

Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems* (Vol. 38, pp. 20004–20031). Neural Information Processing Systems Foundation. https://doi.org/10.52202/085713-0593

Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). *Corrective retrieval augmented generation*. arXiv. https://doi.org/10.48550/arXiv.2401.15884

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., . . . Qiu, Z. (2024). *Qwen2.5 technical report*. arXiv. https://doi.org/10.48550/arXiv.2412.15115

Yoran, O., Wolfson, T., Ram, O., & Berant, J. (2024). *Making retrieval-augmented language models robust to irrelevant context* [Paper presentation]. International Conference on Learning Representations (ICLR 2024). https://doi.org/10.48550/arXiv.2310.01558

Zhang, H., Da, J., Lee, D., Robinson, V., Wu, C., Song, W., Zhao, T., Raja, P., Zhuang, C., Slack, D., Lyu, Q., Hendryx, S., Kaplan, R., Lunati, M., & Yue, S. (2024). *A careful examination of large language model performance on grade school arithmetic* [Paper presentation]. Conference on Neural Information Processing Systems (NeurIPS 2024), Datasets and Benchmarks Track. https://doi.org/10.48550/arXiv.2405.00332

Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., & Zhou, J. (2025). *Qwen3 Embedding: Advancing text embedding and reranking through foundation models*. arXiv. https://doi.org/10.48550/arXiv.2506.05176

Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., & Wen, J.-R. (2025). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, *43*(6), 1–47. https://doi.org/10.1145/3748302

Zhang, Z., Jia, M., Lee, H.-P., Yao, B., Das, S., Lerner, A., Wang, D., & Li, T. (2024). "It's a fair game", or is it? Examining how users navigate disclosure risks and benefits when using LLM-based conversational agents. In *Proceedings of the CHI Conference on*

Human Factors in Computing Systems (pp. 1–26). ACM. https://doi.org/10.1145/3613904.3642385

Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J., & Huang, G. (2024). ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, *38*(17), 19632–19642. https://doi.org/10.1609/aaai.v38i17.29936

Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. *Proceedings of the AAAI Conference on Artificial Intelligence*, *38*(17), 19724–19731. https://doi.org/10.1609/aaai.v38i17.29946

Appendices

Appendix 1: Gantt Chart

Figure A1.1 shows the work plan to the end of November 2026, and Table A1.1 gives the course milestones.

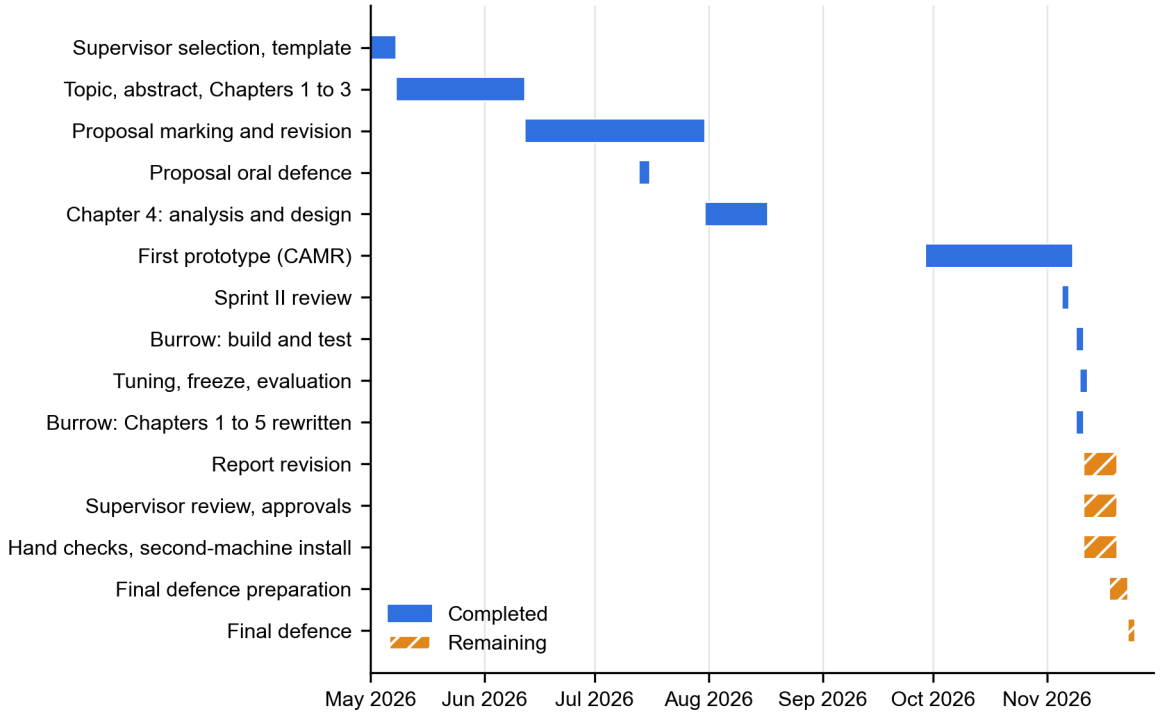


Figure A1.1: Project work plan to the end of November 2026.

Table A1.1: Course milestones and their status

No.	Task	Due date	Status
1	Supervisor selection and allocation	7 May 2026	Done
2	Document formatting and template creation	8 May 2026	Done
3	Topic creation and selection	-	Done
4	Abstract and Chapters 1 to 3	-	Done
5	Proposal submission	12 June 2026	Done
6	Proposal oral defence	13 to 16 July 2026	Done
7	Proposal marking and revision	31 July 2026	Done
8	Chapter 4: System Analysis and Design	17 August 2026	Done
9	Sprint II review	5 November 2026	Done
10	Documentation	Date to be confirmed	In progress
11	Project final defence	23 November 2026	Pending

Appendix 2: Feedback-Resolution Matrix

Table A2.1 records the feedback received and how each item was resolved. Table A2.2 lists every change made to Chapters 1 to 3 since the approved proposal.

Table A2.1: Feedback received and its resolution

Issue	Source	Affected section	Action	Status
"Very hard to understand"	Sprint II review, 5 Nov 2026	1.2, 4.4	The engine was rebuilt around plain files and five named steps, and the report describes it in standard terms	Done
"Just a report of benchmarks"	Sprint II review	1.6, 5.3	A usable app, a terminal chat and an MCP server were built around the engine; the scope now includes them	Done
Show it working with native Ollama models	Sprint II review	5.2, 5.3	Ollama is the only model runner; every local result was produced through it	Done
The person cannot create or remove their own collections	Sprint II review	4.4	The memory is the person's own folder of files; nothing is seeded	Done
Model activity should be visible	Sprint II review	5.3.1	The app states what the engine decided, the method used and the files read under each answer	Done
Can it generate reports natively?	Sprint II review	5.3.1	An answer can be kept as a note; no report format was built	Partly
If it codes, let it code, with live previews	Sprint II review	1.6	The app proposes changes to files for acceptance; live previews were not built	Partly
[PENDING: written proposal comments and defence feedback.]	Proposal defence	-	-	Open

Table A2.2: Changes to Chapters 1 to 3 since the approved proposal

Section	What changed	Why
Whole document	The engine is named Burrow. The old name appears once, in 3.3.	The engine was rebuilt and renamed after the Sprint II review of 5 November 2026. Title and topic are unchanged.
Acknowledgment	"this proposal" became "this project"; thanks to the Sprint II panel added.	Final report.
Abstract	Rewritten second paragraph: vault, five engine steps, Swift, app, terminal chat and MCP server, the conditions compared. One results sentence left pending.	Aligned with what was built.
Keywords	"prompt compression" replaced by "token budget".	The engine budgets what is read; it does not compress prompts.
Abbreviations	Removed CAMR and ANN. Added BM25, FTS5, GSM8K, MCP, RRF, SWE-bench.	Terms used in the revised text.
1.1	"typically between one and three billion parameters" removed from the definition of small models; one sentence added on Gemma 4 and Qwen3.5.	The models hosted are 12 and 4 billion parameters.

Section	What changed	Why
1.2, last paragraph	Solution paragraph describes the vault, the decision whether to use it, the fitted token budget, and notes written with consent. "Screened notes from past interactions" removed.	The memory as built is the person's own files, not only notes from interactions.
1.3.2 (ii)	Retrieval policy "ranks notes by similarity, recency, and importance" became "ranks stored passages by word match and by meaning and follows the links between files".	Retrieval as built is BM25 and cosine merged by reciprocal rank fusion, then a walk over links.
1.3.2 (iii)	"in Swift on an Apple-silicon consumer laptop", "index of the memory store".	Implementation language and platform as built; the database is a rebuildable index beside the vault.
1.4	No change.	Approved wording kept.
1.6	Laptop is a MacBook Pro M5 Pro with 24 GB (was 16 GB). Models named: Gemma 4 12B, Qwen3.5 4B, Claude. Public QA benchmarks replaced by three curated subjects plus GSM8K. Scope sentence now includes the command-line tool, Mac app, terminal chat and MCP server.	As built; surfaces added after the Sprint II review.
1.7.1	Third limitation now says the curated questions were written by a language model.	Honest statement of the evaluation as run.
1.7.2	"repeated timing runs and median latencies" became "seconds taken by every answer". "Several established benchmarks" became "three unrelated subjects and one public benchmark". The optional graph layer is reported as built in a light form.	The plan was one run per question; link walking was built.
2.1	One clause added: systems Burrow is compared with or built to work beside.	Three related works added.
2.2.1	One sentence added on Gemma 4 and Qwen3.5.	Models hosted in the study.
2.2.4	"Memory should be stored densely, as short structured notes" became "what the model reads should be dense, short excerpts and short notes".	The vault keeps the person's files whole; density is applied to what is read.
2.2.5	Engine "mirrors in its index, its retrieval, and its token budgeting". Security design choices became: notes backed by the person's own words, nothing written without consent. LongMemEval cited as 2025 (its conference year).	Provenance and screening were realised as evidence and consent in the Learn step.
2.3	Heading comma corrected ("Memory, Retrieval, and Compression Techniques").	Typing error in the proposal.
2.3.3	Three sentences added on BM25, embedding comparison and reciprocal rank fusion.	Techniques the engine uses, not reviewed in the proposal.
2.4.2	"Adopted directly by this study's retrieval policy" became "shaped the retrieval policy first proposed".	Recency and importance scoring are not in the engine as built.
2.4.4	Last clause: Burrow follows existing links instead of building a graph with a language model.	The associative layer was built as link walking.
2.4.6 to 2.4.8	New: Chroma, Obsidian, Claude Code with the Model Context Protocol, each with one figure from its official source.	The baseline, the form of the memory, and the cloud assistant Burrow works beside.
Figures 2.1 to 2.5	Same images; a source added to each caption.	Course guide: cite figure sources.
Table 2.1	Three rows added.	Three related works added.

Section	What changed	Why
2.5	One sentence added to the first paragraph; "against both a plain vector database and a cloud model" added to the second; "milliseconds" became "seconds".	New related works; times are reported in seconds.
2.6 and Figure 2.9	Independent variable now names the Decide, Fit, Check and Learn steps and the skills. Figure redrawn (was Figure 2.6).	The proposal's figure carried the old name and the old pipeline.
3.1	One sentence added: departures from the proposal are stated.	Course guide: Chapters 1 to 3 aligned with what was built.
3.2.1	PopQA, HotpotQA and 2WikiMultiHopQA replaced by three curated subjects (Kenyan data protection law; software documentation of 2026; a private project). GSM8K kept.	The memory is a personal vault; an encyclopaedia corpus does not represent it.
3.2.2	Sample described: 25 single-fact and 15 multi-fact questions per subject, 50 GSM8K questions by fixed seed; keys checked by script; subjects act as distractors for one another.	As run.
3.2.3	Three conditions became four, with a Chroma baseline.	As run.
3.2.4	Benchmark metrics became accuracy by answer key; bootstrap intervals became an exact sign test; frozen engine added.	As run.
3.3 and Figure 3.1	Four increments plus one optional became five. Increment 3 lists the engine steps. Increment 5 is the app, terminal chat and MCP server. Sentence on the Python prototype and the Swift rebuild. Figure redrawn.	As built.
3.4	Requirements restated for vault, index, Decide, link following, fitted budget, Check, consent. "Single command" became "from the command line".	As built.
3.5.1	Write path, store, read path became vault, index, and the five engine steps; mapping to the proposal's three mechanisms stated; skills added.	As built.
3.5.2	Fourth condition (cloud model with Burrow), plain-search control and Chroma baseline added; rules for an undefined gap and for the private project; engine freeze and written plan added as a second guard. Equation 3.1 kept.	As run.
3.5.3	Task types drawn from curated subjects; paragraph added on the two earlier tests (101 SWE-bench bug fixes, simulated developer agents).	As run.
3.5.4	Exact match and F1 became answer-key accuracy; peak memory and on-disk size removed from the reported measures; sign test named.	As run.
3.5.5	Ablation described as: plain search control, full engine, engine without skills, engine without Check. Pending note on a token budget sweep.	The written plan lists only these ablations.

Section	What changed	Why
3.5.6	Threat list updated: temperature zero, sign test, frozen engine and separate development questions, tested marking rules. Manual audit of graded answers removed. Two remaining threats stated.	As run.
3.6	Tools are now Swift, SQLite with FTS5, Ollama with Gemma 4 and Qwen3.5, Anthropic API, Qwen3 Embedding, Chroma, LangChain, Python (harness only), Git.	As built.
3.7	Deliverable added: application, terminal chat and MCP server. Engine is a Swift package with a command-line tool.	As built.
References	Rebuilt to APA 7 with full author lists and DOIs. Corrected: AgentPoison (five authors, title ends "knowledge bases"), the memory injection paper (Dong and seven co-authors, correct title), Peffers et al. (volume, issue, pages). Added: Anthropic, Chroma, Cormack, Data Protection Act, Gemma 4, Jimenez, LangChain, Model Context Protocol, Obsidian, Ollama, Qwen3.5, Robertson, SQLite, Zhang (Qwen3 Embedding).	APA 7; new sources.

Appendix 3: Further Results

The tables below support Section 5.5. Table A3.1 gives the small model, Table A3.2 the main model by subject, Table A3.3 the paired tests, Tables A3.4 and A3.5 what the engine decided, Table A3.6 where the wrong answers came from, and Table A3.7 one question of each kind. Every answer is saved in the repository under `trials/results/gap-1`, and the full record of the 212 automated tests is in `unit-tests-evaluation-2.log` in the same folder.

Table A3.1: Answers right by set-up and task type, Qwen3.5 4B

Set-up	Knowledge, one fact	Knowledge, several facts	Reasoning
Model alone	5 of 75 (7%)	1 of 45 (2%)	46 of 50 (92%)
Plain retrieval	66 of 75 (88%)	31 of 45 (69%)	9 of 50 (18%)
Vector database (Chroma)	54 of 75 (72%)	21 of 45 (47%)	1 of 50 (2%)
Burrow engine	64 of 75 (85%)	27 of 45 (60%)	49 of 50 (98%)
Cloud model alone	31 of 75 (41%)	9 of 45 (20%)	49 of 50 (98%)
Cloud model with Burrow	71 of 75 (95%)	38 of 45 (84%)	49 of 50 (98%)

Table A3.2: Answers right by subject, Gemma 4 12B and the cloud model

Subject	Alone	Plain retrieval	Chroma	Engine	Cloud alone	Cloud with Burrow
Kenyan law	8 of 40 (20%)	29 of 40 (72%)	17 of 40 (42%)	31 of 40 (78%)	19 of 40 (48%)	33 of 40 (82%)
2026 software	0 of 40 (0%)	32 of 40 (80%)	34 of 40 (85%)	36 of 40 (90%)	14 of 40 (35%)	39 of 40 (98%)
Private project	1 of 40 (2%)	33 of 40 (82%)	24 of 40 (60%)	35 of 40 (88%)	7 of 40 (18%)	37 of 40 (92%)

Subject	Alone	Plain retrieval	Chroma	Engine	Cloud alone	Cloud with Burrow
Arithmetic reasoning	47 of 50 (94%)	7 of 50 (14%)	42 of 50 (84%)	48 of 50 (96%)	49 of 50 (98%)	49 of 50 (98%)

Table A3.3: Paired comparisons by exact sign test

Model	Comparison	Questions	First right only	Second right only	p
Gemma 4 12B	Burrow engine against model alone	Knowledge	95	2	< 0.001
Gemma 4 12B	Burrow engine against model alone	Reasoning	1	0	1.000
Gemma 4 12B	Burrow engine against plain retrieval	Knowledge	10	2	0.039
Gemma 4 12B	Burrow engine against plain retrieval	Reasoning	41	0	< 0.001
Gemma 4 12B	Burrow engine against vector database (chroma)	Knowledge	34	7	< 0.001
Gemma 4 12B	Burrow engine against vector database (chroma)	Reasoning	6	0	0.031
Gemma 4 12B	Burrow engine against engine without skills	Knowledge	0	1	1.000
Gemma 4 12B	Burrow engine against engine without skills	Reasoning	3	1	0.625
Gemma 4 12B	Burrow engine against engine without the check	Knowledge	0	0	1.000
Gemma 4 12B	Burrow engine against engine without the check	Reasoning	0	1	1.000
Qwen3.5 4B	Burrow engine against model alone	Knowledge	86	1	< 0.001
Qwen3.5 4B	Burrow engine against model alone	Reasoning	3	0	0.250
Qwen3.5 4B	Burrow engine against plain retrieval	Knowledge	1	7	0.070
Qwen3.5 4B	Burrow engine against plain retrieval	Reasoning	40	0	< 0.001
Qwen3.5 4B	Burrow engine against vector database (chroma)	Knowledge	29	13	0.020
Qwen3.5 4B	Burrow engine against vector database (chroma)	Reasoning	48	0	< 0.001

Table A3.4: What the engine decided, Gemma 4 12B

Questions	Decision	Count
Knowledge	Search	92
Knowledge	Search, then links	28
Reasoning	Vault left out	50

Table A3.5: The method handed to the model, Gemma 4 12B

Skill	Questions
Answer from documents	92
Work a calculation	36

Skill	Questions
Explain code	28
none	14

Table A3.6: Wrong knowledge answers, by whether the files holding the answer were shown to the model

Model	Set-up	Wrong answers	A needed file was not shown	Every needed file was shown
Gemma 4 12B	Plain retrieval	26	15	11
Gemma 4 12B	Vector database (Chroma)	45	25	20
Gemma 4 12B	Burrow engine	18	8	10
Qwen3.5 4B	Plain retrieval	23	13	10
Qwen3.5 4B	Vector database (Chroma)	45	25	20
Qwen3.5 4B	Burrow engine	29	14	15
Claude	Cloud model with Burrow	11	4	7

Table A3.7: One question of each kind with its answer key

Kind	Question	Key
Law, one fact	Under Kenya's Data Protection Act, 2019, within how many hours of becoming aware of a personal data breach that poses a real risk of harm must a data controller notify the Data Commissioner?	seventy-two or 72
Software, several facts	Across SQLite 3.53.0 and 3.54.0 (both released in 2026), which three new options were added to the sqlite3_limit() interface?	SQLITE_LIMIT_PARSER_DEPTH; SQLITE_LIMIT_SCHEMA; SQLITE_LIMIT_TRIGGER_STEPS
Private project, one fact	In the Burrow project, which named technique merges the word-search ranking with the embedding ranking, and why was it chosen over combining the two scores directly?	reciprocal rank fusion or RRF
Reasoning	The basketball team went to the steakhouse to eat dinner. The first player ate a 6-ounce steak. The second player ate beef tips, containing 8 beef tips, each an ounce in size. The third player ate a one-pound steak. And the fourth and fifth players ordered vegetarian meals. In total, how many ounces of meat were consumed by the team?	30

Before the engine's five steps were built, retrieval alone was tested at tag `evaluation-1`. On 101 real bug fixes from four public repositories, following links reached a changed file on 72 and plain search on 59 ($p = 0.002$). On three held-back synthetic projects worked by simulated developer agents, memory did not measurably help the small model write working code. Those results are saved under `trials/results` and led to the engine described in this report.

Appendix 4: Requirements Traceability

Table A4.1 traces each requirement to the part that implements it and the test that verifies it.

Table A4.1: Traceability from requirement to component and test

Requirement	Objective	Component	Test case	Result
FR-01	iii	`Vault`, `Scanner`, `Chunker`	TC-01	Pass
FR-02	ii, iii	`Retrieval`, `Index`	TC-02	Pass
FR-03	ii, iii	`Retrieval`	TC-03	Pass
FR-04	ii	`Engine.brief`	TC-04, TC-05	Pass
FR-05	ii	`Skills`, `Engine.brief`	TC-06	Pass
FR-06	ii, v	`Fit`	TC-07	Pass
FR-07	iii	`Engine.engage`, `Asking`	TC-08	Pass
FR-08	ii	`Engine.engage`	TC-09	Pass on the command line only
FR-09	ii	`Engine.lessons`, `keep`	TC-10	Pass
FR-10	iii	`Engine.brief`, `BurrowMCP`	TC-11	Pass
NFR-01	iii	`Ollama`	Inspection	Pass
NFR-02	iii	Rebuildable index	TC-12	Pass
NFR-03	iii	`Asking`, `ProjectContent`	Automated tests	Pass on the tools path
NFR-04	iv	Harness	Table 5.6	Pass
NFR-05	iii	`Database`	Inspection	Pass
NFR-06	iii	Swift package	212 automated tests	Pass on one machine

Appendix 5: User Guide

What is needed. A Mac with Xcode and Ollama. Pull the models with `ollama pull gemma4:12b` and `ollama pull qwen3-embedding:0.6b`.

Building. Run `swift build -c release` in the project folder for the command-line tool, and `app/build.sh` for the app. Open `app/build/Burrow.app`.

Using the app.

i. Choose a folder as the vault. An existing Obsidian vault works as it is.
ii. Write notes in the centre pane, and link notes with `[[`.
iii. To add code, choose Add, then Link a Folder.
iv. Open Ask and type a question. Under the answer the app shows what the engine decided, the method it used and the files it read.
v. To have a change proposed, use the change option beside the question field. Review it, then choose Accept or Reject.

Using the terminal.

i. `burrow index <vault> --vault` builds the index.
ii. `burrow fit <vault> --vault --model gemma4:12b` measures the model once.
iii. `burrow skills <vault> --vault --install` adds the starter skills to the vault's `Skills` folder. Edit them or add new ones as Markdown files with a `when:` line.
iv. `burrow ask <vault> "question" --mode engine --model gemma4:12b --vault` answers through the engine.
v. `burrow chat <vault>` opens a chat. `/keep` proposes notes from the conversation and asks before writing each one.

Using Burrow from another model or tool. `burrow brief <vault> "task" --vault --json` returns the prepared prompt for any model. For an MCP client, point it at the built program with the arguments `mcp` and the full path of the vault, and call `vault_prepare`. A client that is a cloud service sends what it reads to that service.
Known limitations. Burrow runs on macOS only. The check with retry runs on `burrow ask`; the app only warns. An empty question and a missing folder are not rejected.

Appendix 6: More Screens of the App

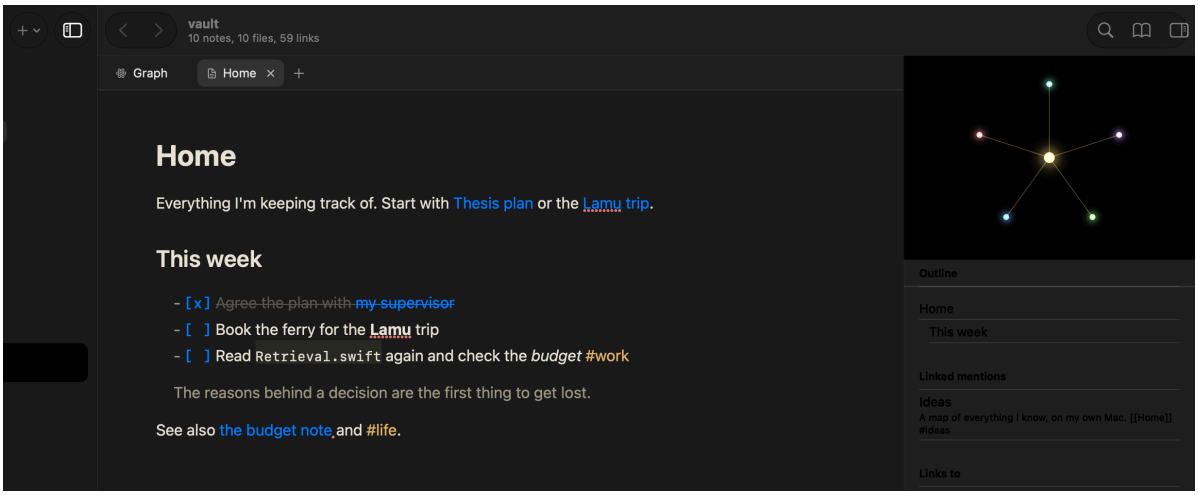


Figure A6.1: A note being edited, with its links and properties.



Figure A6.2: The graph of a vault, coloured by folder.

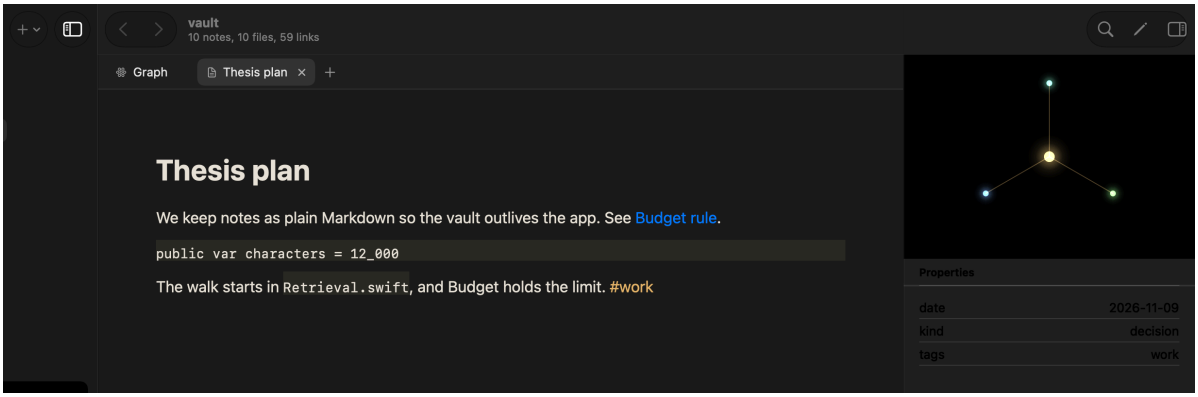


Figure A6.3: A note in the reading view.

Appendix 7: AI-Use Log

Table A7.1 is the AI-use log. Full prompts and replies are retained in the Claude Code session transcripts on the development Mac. [PENDING: the student to add AI use before 9 November and to confirm this log.]

Table A7.1: AI-use log

Date (2026)	Tool and model	Used for	Retained output	Corrections and verification
9 Nov	Claude Code	Plan and scope of Burrow, written with the student	Plan and `DECISIONS.md`	Reviewed and approved by the student
9 Nov	Claude Code	Engine, command line, agent test harness	Source and tests	`swift test`
9-10 Nov	Claude Code	Mac app, terminal view, chat and MCP server	Source and tests	`swift test`; screens reviewed by the student
10 Nov	Claude Code	The earlier evaluation: held-back projects and real bug fixes	Code, projects, results	Plan written before scoring; results saved once
10 Nov	Claude Code	The engine's five steps, skills and faster search, at the student's direction	Source, tests, `DECISIONS.md` D-50 to D-58	212 tests; thresholds set on development questions only
10 Nov	Claude Code agents	Curated documents and questions; evaluation harness; vector database baseline	`trials/gap`	Checker scripts; fixture run; plan written before scoring
10 Nov	Claude (`claude-sonnet-5-5`) through the API	The cloud model in the experiments	Saved answers	Scored by fixed answer keys
10 Nov	Claude Code agents	Chapters 1 to 5 rewritten on the layout of earlier graduates' reports; a verified source pack; diagrams; tables and charts from saved results	Report and generators in `docs/report`	Each source confirmed at its published page; each change from the proposal listed in Table A2.2; tables generated by script
[PENDING]	-	The student's own review and edits of this report	-	-

Appendix 8: Similarity Report

[PENDING: attach the one-page similarity report.]